# A Hybrid Document Understanding System Using Vision and Language Transformers

*MAJOR PROJECT REPORT submitted in partial fulfillment of the requirements for the Award of the Degree of*

**BACHELOR OF TECHNOLOGY**

**In**

**INFORMATION TECHNOLOGY**

**By**


**POOJITHA GATTA 228W1A1287**
**JAYA NAGA VARDHAN LUKKA 228W1A12A4**


*Under the Guidance of*

**Dr. M. Suneetha, Ph.D,**
**Professor and Head**


**DEPARTMENT OF INFORMATION TECHNOLOGY**

**V R SIDDHARTHA ENGINEERING COLLEGE**

**(AUTONOMOUS - AFFILIATED TO JNTU-K, KAKINADA)**

**Approved by AICTE &Accredited by NBA**

**KANURU, VIJAYAWADA-520007**


**ACADEMIC YEAR**

**(2025-26)**

# V.R. SIDDHARTHA ENGINEERING COLLEGE

(Affiliated to JNTUK: Kakinada, Approved by AICTE, Autonomous)
(An ISO certified and NBA accredited institution)
Kanuru, Vijayawada – 520007

## CERTIFICATE

This is to certify that this project report titled **"A Hybrid Document Understanding System Using Vision and Language Transformers"** is a bonafide record of work done by **Poojitha Gatta (228W1A1287) and Jaya Naga Vardhan Lukka (228W1A12A4)** under my guidance and supervision is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, **V.R. Siddhartha Engineering College** (Autonomous under JNTUK) during the year 2025-26.

| | |
|---|---|
| **(Dr. M. Suneetha)** | **(Dr. M. Suneetha)** |
| Dean R&D, Professor & Head | Dean R&D, Professor &Head |
| Dept. of Information Technology | Dept. of Information Technology |

**EXTERNAL EXAMINER SIGNATURE**

*Date of examination:*

# DEPARTMENT OF INFORMATION TECHNOLOGY

# V.R. SIDDHARTHA ENGINEERING COLLEGE

## PROJECT SUMMARY

| S. No | Item | Description |
|---|---|---|
| 1 | **Project Title** | **A Hybrid Document Understanding System Using Vision and Language Transformers** |
| 2 | **Student Names & Numbers** | **POOJITHA GATTA(228W1A1287)**<br>**JAYA NAGA VARDHAN LUKKA(228W1A12A4)** |
| 3 | **Name of The Guide** | **Dr. M. Suneetha, Dean R&D, Professor &Head** |
| 4 | **Research Group** | **Artificial Intelligence and Deep Learning** |
| 5 | **Application Area** | **Text Mining** |
| 6 | **Aim of the Project** | The aim of this project is to develop a hybrid document understanding system that integrates document classification and long document summarization using transformer-based models. |
| 7 | **Project Outcomes** | To build a system that can correctly classify different types of documents by understanding their visual layout and content.<br>To create a summarization model that can read long documents and pick the most important information to generate a short and clear summary. |

**Student Signatures**

1.
2.

**Signature of the Guide**

# ACKNOWLEDGEMENT

First and foremost, I sincerely salute our esteemed institution **V.R SIDDHARTHA ENGINEERING COLLEGE** for giving me this opportunity for fulfilling my project. I am grateful to our Principal **Dr. A.V. RATNA PRASAD**, for his encouragement and support all through the way of my project.

On the submission of this Project report, I would like to extend my honor to **Dr. M. Suneetha**, **Ph.D., Dean R&D, Professor & Head of the Department,** IT for her constant motivation and support during the course of my work.

I feel glad to express my deep sense of gratefulness to my project guide **Dr. M. Suneetha, Ph. D., Dean R&D, Professor & Head of department IT** for **her** guidanceand assistance in completing this project successfully.

I would also like to convey my sincere indebtedness to all faculty members, including supporting staff of the Department, friends and family members who bestowed their great effort and guidance at appropriate times without which it would have been very difficult on my part to finish the project work.

# TABLE OF CONTENTS


| | |
|---|---|
| ACKNOWLEDGEMENT | iii |
| LIST OF FIGURES | v |
| LIST OF TABLES | vi |
| LIST OF EQUATIONS | vii |
| ABSTRACT | viii |
| **CHAPTER-1 Introduction** | **01–10** |
| 1.1 Origin of the Problem | 01 |
| 1.2 Basic definitions and Background | 02 |
| 1.3 Problem Statement | 06 |
| 1.4 Societal Impact | 07 |
| 1.5 Applications | 08 |
| **CHAPTER-2 Review of Literature** | **11–20** |
| 2.1 Description of Existing Systems | 11 |
| 2.2 Summary of Literature Study | 15 |
| 2.3 Software Requirement Specification | 17 |
| **CHAPTER-3 Proposed Method** | **21-39** |
| 3.1 System Architecture | 21 |
| 3.2 Design Methodology | 26 |
| 3.3 Description of Dataset | 28 |
| 3.4 Description of Algorithms | 31 |
| **CHAPTER-4 Results and Observations** | **40-63** |
| 4.1 Stepwise description of Results | 41 |
| 4.2 Models Performance Comparison | 50 |
| 4.3 Observations from work | 60 |
| **CHAPTER-5 Conclusion and Future study** | **63** |
| 5.1 Conclusion | 63 |
| 5.2 Future Work | 65 |
| REFERENCES | xi |
| APPENDIX | |

# LIST OF FIGURES


| Figure | Description | Page |
|---|---|---|
| Figure 3.1 | Architecture Diagram | 31 |
| Figure 4.1 | Evaluation of CodeT5 | 36 |
| Figure 4.2 | Evaluation of BART | 37 |
| Figure 4.3 | Evaluation metrics of Bug Detection | 38 |
| Figure 4.4 | Confusion Matrix | 39 |
| Figure 4.5 | Code generation user interface | 42 |
| Figure 4.6 | Generated Code | 43 |
| Figure 4.7 | User Interface of DebugGPT system | 45 |
| Figure 4.8 | Check Bugs | 45 |
| Figure 4.9 | Code Correction | 46 |
| Figure 4.10 | Generate Report | 47 |
| Figure 4.11 | Generate Test Cases | 47 |
| Figure 4.12 | Performance comparison of BART,LED | 48 |

# LIST OF TABLES


| Table | Description | Page |
|---|---|---|
| Table 2.1 | Literature Review | 12 |
| Table 3.1 | CodeT5 Algorithm | 21 |
| Table 3.2 | BART Algorithm | 22 |
| Table 3.3 | Llama Algorithm | 23 |
| Table 3.4 | Qwen Algorithm | 24 |
| Table 3.5 | DiT algorithm | 25 |
| Table 3.6 | Document summarization | 26 |
| Table 3.7 | RVL-CDIP dataset | 28 |
| Table 4.1 | Evaluation metrics of document classification | 40 |
| Table 4.2 | Performance Comparison of Summarization Models | 41 |

# LIST OF EQUATIONS


| Equation No | Brief Description | Page |
|---|---|---|
| Eq - (1) | Patch Embedding | 18 |
| Eq - (2) | Positional Encoding | 19 |
| Eq - (3) | Attention Mask | 19 |
| Eq - (4) | Document Embedding Generation | 20 |
| Eq – (5) | Chunk Encoding | 20 |
| Eq – (6) | MMR and Sentence Generation | 21 |
| Eq – (7) | Image Preprocessing | 25 |
| Eq – (8) | Patch Tokenization | 26 |
| Eq – (9) | Document Chunking | 27 |

# ABSTRACT

The increasing use of digital documents such as research papers, reports, and business documents necessitates the development of a robust automatic system that possesses intelligence to understand and summarize those documents automatically. Manual processing of many pages of documents is not efficient as well as time-consuming. It is particularly difficult when long and complex textual information is involved. Traditional NLP models are unable to process long documents effectively, remove redundancy, and maintain coherent context throughout text. There are several difficulties, including long sequences of texts, semantic identification of the document, and other similar challenges. Here, we discuss a hybrid approach for the document classification and summarization task. In document classification, a Vision Transformer based document image Transformer DiT will be used. For document summarization, two transformers-based models are considered which are Bidirectional and Auto-Regressive Transforme BART and Longformer Encoder-Decoder (LED). The key application areas of the proposed work are enterprise document management, research paper summarization, digital libraries, legal and financial document analysis, healthcare record processing, automated report generation, knowledge management system, and other similar tasks where AI document processing is involved. The classification accuracy of the framework is achieved up to 99%. For document summarization, the LED model performs more effectively than the BART model because of its higher values for semantic accuracy, 0.851, and layout fidelity index, 0.937. The suggested architecture provides an effective way of automatic understanding and document summarization.



**KEYWORDS:** Document Classification, Long Document Summarization, Vision Transformers, Document Image Transformer (DiT), Bidirectional and Auto-Regressive Transformers (BART), Longformer Encoder–Decoder (LED).

# CHAPTER-1

# INTRODUCTION

This chapter provides an introduction to the basic principles and motivation for developing the document understanding system. This chapter gives an overview of the background information pertaining to the document classification, document summarization, and transformer-based approach. In this chapter, the problem statement will be defined along with the deficiencies of existing solutions and justification for using the proposed system. The chapter describes the aims of the project and the possible application areas.

## 1.1 ORGIN OF PROBLEM

The increasing use of digital documents in various industries has led to the problem of efficient data processing and analysis. One of the biggest problems that exist in modern solutions for this purpose is poor scalability, especially with documents that require detailed analysis. This refers to large-scale documents, such as academic papers, reports, articles, manuals, and other types of documents with complex layouts. The presence of a large number of text, images, figures, tables, and other content makes documents harder to analyze.

Recently, some new approaches were proposed using the principles of machine learning and deep neural networks. Vision and language models have proven useful for analyzing both textual and graphical content. On the other hand, transformer models are now widely used to analyze long texts. Despite progress made, most existing solutions remain limited regarding their ability to analyze long documents effectively[1].

The above-discussed constraints underscore the necessity for adopting an efficient approach capable of solving both classification and summarization problems through a combined strategy. In current methods, efforts are mostly focused on one side of the equation, and hence, the process becomes very inefficient during practical implementation. Another critical constraint is the balancing of the speed of processing, its efficiency, and the conservation of document structure during classification and summarization[14].

In order to overcome the above-discussed constraints, this project adopts a hybrid document analysis technique, which employs both the processes of document classification and summarization. The adopted vision-based transformer architecture will be helpful in identifying document layout, while the advanced language models will assist in generating relevant summaries of long texts. This project, therefore, intends to integrate both techniques under one framework for improved efficiency and performance[6].

## 1.2 BASIC DEFINITIONS AND BACKGROUND

### 1.2.1 Vision Transformer(ViT):

Vision Transformer (ViT) is a deep learning architecture that applies the transformer framework, originally developed for natural language processing, to image data. In this approach, an image is divided into fixed-size patches, and each patch is treated as a token similar to words in a sentence. These tokens are then converted into embeddings and combined with positional encodings to preserve spatial information. The sequence of embedded patches is passed through transformer encoder layers, where self-attention mechanisms help the model understand relationships between different regions of the image.

A key strength of Vision Transformers is their ability to capture global context, as self-attention allows every patch to interact with every other patch in the image. This enables better understanding of long-range dependencies compared to traditional convolutional neural networks (CNNs), which focus on local features. However, ViTs generally require large datasets for effective training due to the lack of inherent inductive biases found in CNNs. Despite this, they have achieved state-of-the-art performance in various computer vision tasks, making them a powerful approach for modern image analysis.

### 1.2.2 Document Image Transformer(DiT):

The Document Image Transformer (DiT) is a specialized adaptation of Vision Transformers designed for document understanding tasks. It focuses on extracting layout-aware features such as text alignment, spacing, font patterns, and overall document structure rather than relying solely on textual content. By dividing a document image into patches and treating them as tokens, DiT captures both visual and positional information. This enables the model to understand how different elements like headings, paragraphs, tables, and images are organized within a document.

A key advantage of DiT is its use of self-attention to learn spatial relationships across the entire document. This allows it to capture long-range dependencies and recognize patterns in structured and semi-structured documents effectively. As a result, DiT performs well in tasks such as document classification and layout analysis, especially when documents share similar formats. Despite requiring large datasets and computational resources, DiT provides a powerful and efficient approach for modern document understanding systems[2].

### 1.2.3 Bidirectional and Auto-Regressive Transformer(BART):

BART is an encoder–decoder architecture based on transformers that is widely used for tasks such as text summarization. It employs a bidirectional encoder to understand the full context of the input text and an autoregressive decoder to generate output sequences step by step. The model is trained using a denoising objective, where corrupted input text is reconstructed into its original form, enabling it to learn strong contextual and linguistic representations. This makes BART highly effective in producing coherent, fluent, and grammatically correct summaries.

In addition, BART performs well in abstractive summarization, where it generates new sentences rather than simply extracting portions of the original text. Its ability to capture long-range dependencies and contextual relationships allows it to produce meaningful summaries that retain the core information of the input. However, BART has limitations when handling very long documents due to its fixed input length, which may lead to information loss in large texts. Despite this, it remains a powerful and widely used model for summarization and other natural language generation tasks due to its flexibility and strong performance.[3]

### 1.2.4 Longformer Encoder-Decoder(LED):

The Longformer Encoder-Decoder (LED) is an advanced transformer model designed to efficiently process long documents by using sparse attention instead of full self-attention. This mechanism reduces computational complexity while allowing the model to handle significantly longer input sequences. By combining local and global attention patterns, LED can capture relationships between both nearby and distant tokens, ensuring that important contextual information is preserved across the entire document.

This capability makes LED highly effective for long document summarization tasks, such as research papers and reports, where maintaining context and coherence is essential. Unlike traditional models with limited input capacity, LED generates more comprehensive and structured summaries by understanding long-range dependencies. As a result, it produces summaries that are not only informative but also maintain logical flow and overall document structure.

### 1.2.5 Maximum Marginal Relevance(MMR):

Maximal Marginal Relevance (MMR) is a selection technique used in extractive summarization to identify sentences that are both relevant to the main document and non-redundant with respect to each other. It works by selecting sentences that best represent the core content while avoiding repetition, ensuring that the summary remains informative and

concise. Instead of choosing sentences solely based on importance, MMR balances relevance with diversity, allowing the summary to cover different aspects of the document effectively.

The method operates iteratively by selecting sentences that maximize similarity to the document while minimizing similarity to already selected sentences, often using measures like cosine similarity. This helps in reducing redundancy and improving content coverage, especially in long or complex documents. As a result, MMR enhances the quality of summaries by ensuring that each selected sentence contributes unique information, making it a valuable technique in modern summarization systems[11].

### 1.2.6 CodeT5:

CodeT5 is a transformer-based model that is dedicated explicitly to programming. It is a specialized version of the T5 model which is fine-tuned for programming purposes on large code datasets in different programming languages. This model can undertake a variety of tasks concerning coding including code summarization, code translation, and code generation. Its architecture is appropriate for the analysis of code syntax and semantics, making it an effective solution to translate natural language instructions to code [2].

In addition, CodeT5 incorporates code-specific pretraining objectives such as identifier-aware learning, which helps the model better understand variable names and programming structures. This enables it to capture deeper relationships within the code, improving both accuracy and contextual understanding. As a result, CodeT5 demonstrates strong performance in generating syntactically correct and semantically meaningful code, making it highly suitable for applications like automated programming assistance, bug fixing, and intelligent code completion systems.

### 1.2.7 LLaMA-3.3-70B Versatile:

LLaMA-3.3-70B Versatile is a large transformer-based language model that is meant to carry out deep reasoning, code understanding, and the generation of structured texts. LLaMA-3.3-70B Versatile processes input tokens in order to build relations between pieces of code, variables, and program flow. This model makes use of self-attention to process important elements in the input, followed by the usage of a feed-forward network to produce appropriate predictions. Being highly parametrized, the model is capable of recognizing patterns, spotting logic mistakes in code, and reasoning through code structure.

### 1.2.8 Qwen3-Next-80B:

Qwen3-Next-80B is a large-scale transformer architecture designed for advanced reasoning, instruction-following, and multilingual understanding, with strong capabilities in code-related tasks. It leverages deep attention mechanisms to analyze code structure, detect naming inconsistencies, identify semantic errors, and understand logical flow within programs. This enables the model to not only generate code but also provide meaningful explanations and corrections, making it highly useful for debugging and code refinement tasks.

In addition, Qwen3-Next-80B is optimized for instruction-based interactions, allowing it to respond accurately to user prompts across diverse domains. Its ability to handle complex queries and generate well-structured outputs such as reports, summaries, and corrected code enhances its practical applicability. Although it performs well in many scenarios, its effectiveness may vary when dealing with highly complex logical errors compared to more specialized models. Nevertheless, its versatility, strong reasoning ability, and multilingual support make it a powerful tool for modern AI-driven development and analysis tasks.

### 1.2.9 Evaluation Metrics:

The following evaluation metrics are used for evaluating the performance of systems of code generation, bug detection and document understanding systems:

**Semantic Accuracy:**

Semantic accuracy is used to measure how accurately the summary captures the original meaning of the article. The main purpose is to ensure that all necessary pieces of information are covered. A high level of semantic accuracy proves that the summary stays true to its source. This metric plays an essential role in assessing summaries.

**Semantic Consistency Score (SCS):**

Semantic Consistency Score estimates the logical consistency of the summary compared to the article. It also analyzes the smoothness of transitions between pieces of information. A high score means a better consistency of information presented in the summary. It guarantees that there will be no contradictions.

**Redundancy:**

Redundancy is used to measure how repetitive the information is in the summary. High levels of redundancy make the summary less effective. The fact of unnecessary repetition can indicate problems with generating new data.

**Information Score:**

Information score refers to the amount of information that is contained in the original

source. Information score helps in checking whether the summary contains all the relevant information about the original text. A high information score shows that the summary contains all the required information.

**Layout Fidelity Index (LFI):**

Layout Fidelity Index is used to check the layout of information in the summary. It measures the way information has been organized. A high LFI means that the information is represented properly. It is very important for documents that have a complex format.

**Bilingual Evaluation Understudy(BLEU):**

Calculates the extent to which n-grams in the created code align with the reference code. The higher the score, the more similar they are.

**Recall-Oriented Understudy for Gisting Evaluation(ROUGE):**

Calculates overlap between the generated and the reference texts through the use of unigram overlap (ROUGE-1), bigram overlap (ROUGE-2), and longest common subsequence overlap (ROUGE-L).

**METEOR:**

Looks at precision and recall as well as synonym matching to measure language similarity beyond what is provided by BLEU.

**BERTScore (F1):**

Determines the semantic similarity of the generated code relative to the reference code using contextualized word embeddings provided by pre-trained BERT models.

## 1.3 PROBLEM STATEMENT

The proliferation of digital documents in different fields has posed serious problems in processing and obtaining valuable information. The generation of documents like research articles, reports, invoices, and other business documents takes place in huge numbers. The conventional approaches towards document processing and analysis have been quite cumbersome and labor-intensive. The current systems lack the ability to process the complicated layout of documents, including tables, pictures, and formats.

Furthermore, most of the contemporary methodologies face a lot of difficulty in achieving satisfactory performance while handling lengthy documents. Consequently, these systems tend to be costly and inefficient due to the increased computational complexity. In general, almost all current approaches concentrate on document classification or summarization, yet none of them addresses both processes collectively. It becomes difficult for such algorithms to be used practically as they cannot perform both classification and summarization simultaneously.

Moreover, semantic coherence and retention of structure in long document summarization are still unresolved issues. Besides, there is always a necessity to maintain semantic cohesion in the processed output document and avoid redundancy. Therefore, the development of a robust document processing and summarizing system with accurate classification capabilities has become crucial.

### 1.3.1 Objectives:

To develop a system which is capable of distinguishing between different document types based on visual and content characteristics.

To develop a summarization model that takes into account lengthy documents and is able to extract significant information from them for generation of concise summaries.

### 1.3.2 Outcomes:

A document classification system capable of predicting document types irrespective of complexity of layout design.

A summarization system which generates concise, significant, and redundant free summaries of lengthy documents.

## 1.4 SOCIETAL IMPACT OF PROPOSED WORK

With the increase in digitalization in almost all sectors, the system will help efficiently deal with and analyze large digital texts. The automatic classification and summarization of digital texts will ensure that users get the information that is needed without reading the complete text. Thus, knowledge access and learning efficiency will be enhanced by the use of the proposed system.

Decisions in sectors where accurate and timely decisions are necessary can be made by using the proposed system. Professionals will have accurate classification and summaries of information presented in documents. They can make decisions more easily while avoiding human error.

The system provides a solution to the challenge in the industries undergoing digitalization where there is a need for manual processing of digitalized information. This reduces errors due to time wastage, inconsistency, and human error. Thus, efficiency and productivity are improved in the industries undergoing digitalization.

The proposed system promotes inclusion in accessing the available knowledge base. Students or other users who do not have time to read all the texts can easily understand the information by using the system.

## 1.5 APPLICATIONS

**Research and Academic Analysis:**

This system will play a crucial part in academic settings due to its ability to classify and summarize academic papers, research journals, conferences, and other technical documents. Students and researchers find it challenging to go through many literature documents during their limited time period. Such a system makes it easy for people to conduct their literature review through summarizing vital points in less time. They can also discover some knowledge through recognizing patterns in different research documents.

**Business Documents Management:**

There are numerous unstructured data records that are produced every day within business organizations through means such as invoices, agreements, emails, reports, and minutes of meetings. The new system will allow these documents to be automatically classified and summarized, thus facilitating their access and use when making decisions. This will reduce manual work, increase efficiency, and prevent the loss of important information.

**Legal and Financial Document Management:**

For lawyers and finance experts, the process involves handling long and complicated documents like contracts, agreements, audit reports, and financial records. The software helps to extract key points from these documents, create summaries of long legal texts, and identify important points about the finances. It saves a lot of time that would have been taken to analyze documents. It is helpful for both lawyers, auditors, and finance experts.

**Medical Documentation Management:**

Many types of data, including patient files, diagnosis details, prescriptions, and case studies, are generated by healthcare organizations. The suggested system will be able to sort and synthesize all the information about medical documents, providing fast access to necessary patient information for healthcare specialists. As a result, better clinical decision making will occur, faster diagnosis will be provided, and improved patient care will be ensured.

**Government and Administrative Services:**

Government agencies deal with numerous documents like policies, reports, records, and others. The use of the system can greatly facilitate the process of classification and abstraction of documents, resulting in effective information access. It promotes transparency and helps improve the performance of public services, while at the same time assisting in policy analysis through document summarization.

**News Aggregation and Media Analysis**

It is possible to apply this system on news portals to classify and summarize news articles obtained from different sources. This enables the readers to know what is happening around the world without having to read through full news articles. Media houses can also use the system for analysis purposes.

**Customer Support and Email Management**

There are numerous queries raised by customers that organizations get on a daily basis. There is the capability to categorize these queries and generate a summary of them. This will help the organization respond to their customers better, prioritize some requests and improve customer satisfaction.

**Educational Content Management**

Such a system can be employed by educational organizations to summarize information from various textbooks, lectures, and other learning material. Students can thus gain insights into critical concepts in record time, thus enhancing their efficiency in learning.

**Enterprise Knowledge Management**

Larger companies hold large amounts of internal documents, reports, and information. Information can be arranged by the system in an efficient manner that makes it easy for the employees to refer to the knowledge base when necessary.

**Social Media and Content Analysis**

This system can process vast amounts of content from social networking sites, blogs, online forums, etc., thus facilitating the identification of trends, sentiments, and topics of interest.

In summary, the motivation behind the proposed document understanding system is provided. The problem of having too many digital documents and dealing with difficulties in efficient processing of lengthy and complex documents has been mentioned. The main problem areas include time consuming, information overload, and lack of effective extraction of useful data from the document.

This chapter has also mentioned different approaches and the problems related to them. Many models either traditional models or deep learning models only address single issues and do not present any integrated approach towards document understanding. The problems include dealing with long documents, context understanding, and multiple task integration like classification and summarization.

Another issue discussed in the chapter is the requirement of hybrid models in which classification and summarization are combined together to achieve a better solution. The

proposed approach would make the process more efficient, effective, accurate, scalable, and would minimize human labor.

Finally, this chapter presents the proposed approach and explains its importance in solving current problems and providing an integrated solution to the problems in intelligent document processing.

# CHAPTER – 2
# REVIEW OF LITERATURE

The discussion in this chapter is based on an extensive literature analysis of the research papers gathered related to the concepts of document understanding, classification, and summarization. This literature analysis helps to explore the methodologies, approaches, and techniques used by the various authors concerning the implementation of deep learning techniques for the effective understanding, classification, and summarization of documents.

In addition, an evaluation of various models has been provided in terms of advantages and disadvantages, as well as their applications in different scenarios. Various advancements including the use of multimodal learning techniques that take into account the use of visual and textual information for better outcomes have been analyzed in the context of the current state of the art of this technology.

In particular, there is a focus on how the classification and summarization can be done effectively for long texts that are difficult to summarize otherwise. The information discussed in this chapter not only provides the justification for implementing this kind of system but also gives some ideas to work on it efficiently.

## 2.1 DESCRIPTION OF EXISTING SYSTEM

**M-DocSum: Do LVLMs Genuinely Comprehend Interleaved Image-Text in Document Summarization**

**Authors:** Haolong Yan, Kaijun Tan, Yeqing Shen, Xin Huang, Zheng Ge, Xiangyu Zhang, Si Li, Daxin Jiang

**Year:** 2025

**Observations:**

The focus of this research work is to assess how effective LVLMs can perform the task of understanding and producing summaries of documents with interleaving of texts and images. In this context, researchers have developed a new benchmark named as M-DocSum-Bench, which aims at analyzing the multimodal summarization capabilities of LVLMs through real-life scientific papers. One important point highlighted by researchers is the problem faced by existing models in terms of maintaining consistency, linking visual features with the corresponding text, and managing long-range dependencies among various parts of documents. Moreover, issues related to hallucination, inadequate visual grounding, and confusion regarding similar images are also discussed. M-DocSum-7B is a baseline model introduced for improvement[1].

**DocRefine: An Intelligent Framework for Scientific Document Understanding and Content Optimization**

**Authors**: Kun Qian, Wenjie Li, Tianyu Sun, Wenhong Wang, Wenhan Luo

**Year:** 2025

**Observations:**

In this paper, we describe DocRefine, an approach aimed at intelligent comprehension and optimization of scientific papers. The system makes use of a multi-agent architecture that comprises agents for layout recognition, multimodal interpretation, instructions splitting, and summarization. The system guarantees high precision and consistency in its output thanks to the closed loop feedback process it employs. DocRefine successfully processes challenging aspects of scientific papers such as tables, images, formulae, and references within PDF files. Our experiments revealed that the proposed approach is superior to previous solutions when applied to such document-related tasks as editing, summarization, and content refinement[2].

**Coordinate Matrix Machine: A Human-Level Concept Learning to Classify Very Similar Documents**

**Authors:** Amin Sadri, M. Maruf Hossain

**Year:** 2025

**Observations:**

A model called the Coordinate Matrix Machine (CM2) is introduced in this study as a document classification system with awareness of document structure and the ability to learn concepts like humans. Rather than depending only on textual data, this machine employs the spatial structure along with coordinate information to understand the document structure. This intelligent machine has been developed to classify documents accurately with very little training data available, sometimes just a single example per category (One-Shot Learning). Not only is computation cost reduced here, but the burden of acquiring large volumes of training data is also done away with. This method has proven to be efficient and explainable for practical purposes like financial document classification[3].

**Summarization-Based Data Augmentation for Document Classification**

**Authors:** Yueguan Wang, Naoki Yoshinaga

**Year:** 2024

**Observations:**

This paper proposes a new technique for data augmentation, which is named SUMMaug, to enhance document classification tasks. In particular, the technique is used to generate summaries of documents that lead to more simple and informative instances. The technique makes use of

curriculum learning, which refers to the initial training of models using the summarized easy version of documents before presenting them with the entire document. This leads to better learning of complicated structures. The experiments conducted demonstrate that the method achieves better performance than previous augmentation approaches.[4]

**VisFormers—Combining Vision and Transformers for Enhanced Complex Document Classification**

**Authors:** Subhayu Dutta, Subhrangshu Adhikary, Ashutosh Dhar Dwivedi

**Year:** 2024

**Observations:**

In this paper, the VisFormers framework is proposed which makes use of vision-related features combined with transformers to classify documents. OCR is used to acquire textual information from the document and merge it with visual features like layout, structural format, etc. A multi-headed architecture is used in the model that takes both types of information into account while training. Experiments demonstrate that the model shows far better accuracy than traditional frameworks such as VGG19 and basic transformers. The model performs exceptionally well on documents with complex structures having similarities in appearance [5].

**LNLF-BERT: Transformer for Long Document Classification with Multiple Attention Levels**

**Authors:** Linh Manh Pham, Hoang Cao The

**Year:** 2024

**Observations:**

The proposed method is called LNLF-BERT. It is a transformer-based approach explicitly aimed at long-document classification tasks. In particular, the proposed solution uses a hierarchical attention approach involving two levels – sentence-level attention and global document-level attention. The two-level attention approach allows overcoming the limitations of BERT approaches when classifying long documents. This issue arises because of the extremely high computational cost associated with using full self-attention. In contrast, LNLF-BERT provides a good balance between capturing context and using fewer resources. The results are comparable to those achieved by state-of-the-art approaches such as Longformer and BigBird [6].

**Domain-Specific Multi-Document Political News Summarization Using BART and ACT-GAN**

**Authors:** Zekai Nie, Masooma Arshad, Syed Khaldoon Khurshid, Abqa Javed, Talha Waheed, Qianqian Zhang, Muhammad Farrukh Shahzad

**Year:** 2025

**Observations:**

In this paper, we will describe a new framework that employs a set of transformer based models to produce summaries for several political news articles. In this regard, BART is used alongside the ACT-GAN algorithm to create artificial data which belongs to the target domain. The proposed framework allows for processing several texts at once and creating a unified coherent summary. Experiments proved that BART performed significantly better than other existing summarizers such as T5, Pegasus, and Seq2Seq. This proves the significance of the use of domain-specific training data and synthetic data augmentation to improve performance. However, this technique requires an extra effort for dataset creation [7].

**CoTHSSum: Structured Long-Document Summarization via Chain-of-Thought Reasoning and Hierarchical Segmentation**

**Authors:** Xiaoyong Chen, Zhiqiang Chen, Shi Cheng

**Year:** 2025

**Observations:**

CoTHSSum framework for generating summaries of lengthy documents using the mechanism of chain-of-thought reasoning along with the process of hierarchical segmentation is presented in this paper. Hierarchical segmentation allows breaking the documents into small segments while the generation of summaries takes place in stages until a final summary is obtained. The structured reasoning process allows the elimination of hallucinations from the summaries. Moreover, the generation of hierarchical summaries facilitates the inclusion of important information while preserving logical coherence in the generated summaries. Experimental findings indicate that CoTHSSum performs better than other summarization methods available today [8].

## 2.2 SUMMARY OF LITERATURE STUDY

This section provides an exhaustive review of the current literature on document understanding, classification, and summarization, especially emphasizing recent developments in transformer-based and multimodal methods. Due to the exponential growth in the volume of digitized documents, it becomes increasingly necessary to develop intelligent systems that can process textual as well as visual data effectively. Recent research highlights the importance of LVLMs and transformers in tackling the issues associated with complicated document layouts and lengthy textual information. Such methods have proven highly effective compared to conventional techniques [15][20].

Document understanding using multiple modes has attracted a lot of interest, with both textual and graphical content being processed together. The M-DocSum paradigm studies the

capacity of LVLMs in understanding the interleaved structure of text and images to generate summaries. Nevertheless, the paradigm points out problems that may arise while ensuring consistency when dealing with long documents [16]. On the other hand, the DocRefine framework proposes a multi-agent cooperative system by utilizing layout analysis and language models to improve document understanding and summarization. While the framework boosts semantic and layout precision, it introduces high computational costs because of its multi-agent design [10].

With regards to document classification, there have been some breakthroughs made in layout-aware techniques. The Coordinate Matrix Machine (CM2), which makes use of the spatial information within documents, classifies documents very accurately without needing much training data and hence is highly efficient and interpretable [3]. Another method that has yielded good results for document classification is summary-based augmentation. Through this technique, noise is minimized and enhanced feature representation of the data is possible, which boosts the robustness of transformers and makes them more generalized [4]. Lastly, hybrid systems called VisFormers can be used to classify documents with the same text but different layouts efficiently [9].

The processing of large documents continues to be a key problem for document processing systems. Solutions like LNLF-BERT tackle this problem through the use of hierarchical attention models that incorporate information from sentence and document levels, allowing efficient processing without compromising efficiency [12]. Regarding the task of document summarization, transformer models such as BART with ACT-GAN help generate improved summaries using the power of artificial data through adversarial learning, producing coherent and contextually relevant summaries [7]. In addition, CoTHSSum uses chain-of-thought and hierarchical partitioning methods for generating logically sound and accurate summaries for complicated documents [13].The summary of literature review is shown in Table 2.1 .

***Table 2.1: Literature Review***

| Sno. | Title | Publication Details | Algorithms Used | Dataset | Application & Research Gap |
|---|---|---|---|---|---|
| 1 | M-DocSum: Do LVLMs Genuinely Comprehend Interleaved Image-Text in Document Summarization | Year: 2025, Journal: arXiv preprint | LVLMs, Multimodal Summarization, Benchmark (M-DocSum-Bench) | M-DocSum-Bench (ArXiv papers dataset) | Focuses on improving multimodal document summarization by integrating image-text understanding; however, existing models struggle to effectively align visual |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | and textual features, leading to limited performance improvements |
| 2 | DocRefine: Intelligent Framework for Scientific Document Understanding | Year: 2025, Journal: IEEE (Conference/Preprint) | Multi-agent system, LVLMs, Layout analysis, Summarization | DocEditBench dataset | Aims to enhance document understanding, editing, and summarization using multi-agent collaboration; despite improved semantic accuracy, it requires high computational resources, limiting scalability |
| 3 | Coordinate Matrix Machine for Document Classification | Year: 2025, Journal: Springer | Structure-based learning, Spatial layout analysis, One-shot learning | Structured document datasets (bank statements, templates) | Designed for efficient classification of visually similar documents with minimal training data; however, it lacks robustness when applied to unstructured or highly diverse document formats |
| 4 | Summarization-Based Data Augmentation for Document Classification | Year: 2024, Journal: Elsevier (Information Processing domain) | SUMMaug, Curriculum learning, Transformer models | IMDb dataset | Improves document classification by using summarized data for training; however, summarization may remove important fine-grained contextual details, affecting classification performance |
| 5 | VisFormers—Combining Vision and Transformers for Document Classification | Year: 2024, Journal: IEEE Transactions on Multimedia | OCR, Vision + Transformer hybrid model, Multi-headed architecture | RVL-CDIP dataset | Enhances classification accuracy by combining visual and textual features; however, increased model complexity leads to higher computational cost and resource requirements |
| 6 | LNLF-BERT: Transformer for Long Document Classification | Year: 2024, Journal: ACM Transactions on Information Systems | Two-level attention (sentence + document), Transformer | Long text classification datasets | Efficiently handles long document classification by balancing local and global context; |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | however, it still faces challenges with extremely long documents and memory efficiency |
| 7 | Political News Summarization using BART and ACT-GAN | Year: 2025, Journal: IEEE Access | BART, ACT-GAN, Transformer models | Synthetic political news dataset | Generates coherent summaries from multiple political news sources; however, reliance on synthetic datasets limits real-world applicability and generalization |
| 8 | CoTHSSum: Long Document Summarization using Chain-of-Thought | Year: 2025, Journal: arXiv preprint | Chain-of-Thought reasoning, Hierarchical segmentation, LLMs | Multiple datasets (scientific, legal, biomedical texts) | Improves long document summarization using structured reasoning and hierarchical segmentation; however, it introduces higher processing complexity and increased latency |

In summary, there is a clear trend towards intelligent and integrated document understanding that leverages multimodal learning, transformers, and reasoning mechanisms. Although these models achieve better performance in terms of classification and summarization, they still face issues like managing very long documents, ensuring consistency among multimodal inputs, and minimizing computation cost. Such constraints underscore the demand for a more effective solution, forming the main impetus behind the hybrid system proposed in this project.

## 2.3 SOFTWARE REQUIREMENT SPECIFICATION

### 2.3.1 Purpose

The goal of this research is to design and develop an intelligent document understanding system that can automatically classify documents and create summaries using modern deep learning and transformer-based models. The main idea is to eliminate human participation in handling huge numbers of documents to boost effectiveness, precision, and speed.

With the advent of digital technologies, companies and people face the problem of vast amounts of both structured and unstructured data, and its manual processing takes a lot of time. The presented approach solves this problem by automating document processing including document classification and summarizing.

As for the classification of documents, vision transformer-based models are used, which makes possible the identification of visual structure of document, its layout, and other characteristics. To create document summary, transformers such as BART and LED are used due to their superior results.

The purpose of the system described above is not only to eliminate human intervention in document processing but also to assist decision making.

### 2.3.2 Scope

This application allows users to input their documents using a simple user interface in the form of a website, where the user can either upload files (such as PDF or image files) or enter the textual content itself into the application for processing. This is done by implementing a combined pipeline of pre-processing the document, classification of the document using visual models, and summarization of the document using transformers.

The proposed solution will have applications for more than one functionality that includes classifying documents, summarizing long documents, pre-processing both structured and unstructured data, and processing large documents efficiently. This software is intended to be used in numerous fields that include academics, businesses, law firms, and hospitals. The program will ensure scalability to enable it to process numerous documents at once without compromising performance and accuracy.

### 2.3.3 Intended Audience

- Scholars and students who require rapid comprehension of vast quantities of academic documentation.
- Businessmen and analysts dealing with corporate, legal, and financial documentation.
- Organizations dealing with massive storage of documentation.
- Data Scientists and developers creating AI-powered document processing systems.
- Government employees managing governmental documentation and policy reports.

### 2.3.4 Functional Requirements

**FR1:** The system must enable the uploading of documents (in PDF/image/text format) or enter textual data via the user interface.

**FR2:** The system must pre-process the entered data by scaling, normalizing, and formatting the documents to be fed into the models.

**FR3:** The system must be able to categorize documents based on predetermined classes using the vision transformer model (DiT).

**FR4:** The system must summarize lengthy documents into brief yet meaningful summaries using transformer models like BART and LED.

**FR5:** The system must capture information and structure while summarizing documents.

**FR6:** The system must present classification outcomes and document summaries in easily comprehensible form to the user.

**FR7:** The system must effectively deal with multiple documents.

**FR8:** The system must appropriately integrate the classification and summarization processes.

### 2.3.5 Non-Functional Requirements

**NFR1:** System response time will be optimized such that the results can be generated in a reasonable amount of time (10-15 seconds for normal input).

**NFR2:** System accuracy in classification and summarization will be maximized.

**NFR3:** Application scalability will be maximized to deal with huge amounts of data and multi-user interactions.

**NFR4:** System reliability and consistency will be maximized for different document types.

**NFR5:** User interface simplicity and interactivity will be optimized for usability.

**NFR6:** System security will be optimized to preserve user data throughout uploading and processing phases.

**NFR7:** System maintainability will be optimized to facilitate future integration of better models and techniques.

**Additional Non-Functional Requirements**

**NFR8:** The system should ensure high availability with minimal downtime.

**NFR9:** The system should be compatible across multiple platforms (web/mobile).

**NFR10:** The system should be optimized for computational efficiency.

**NFR11:** The system should ensure fault tolerance and error handling mechanisms.

**NFR12:** The system should support modular architecture for easy upgrades.

This section presents an analysis of the current state of research into document understanding, classification, and summarization with the use of advanced deep learning and transformer-based methods. This review discusses a range of approaches, algorithms, and techniques proposed by other researchers in prior studies, and their contributions and advantages. According to the reviewed literature, there is a tendency for models based on transformers and language models to be used more widely. Multimodal approaches utilizing both text and visual data provide better results when working with documents containing images and tables.

In recent years, research has also emphasized the need for contextual and structural data extraction from the document itself. The latest approaches have focused on the pattern

recognition of the layout, the spatial arrangement, and the hierarchy of the document rather than the textual part alone. Models that incorporate the structure of the document, hybrid vision-transformer model, and the application of attention-based models can assist the system in recognizing key areas of the document and enhancing the accuracy of classification and summarization.

Despite all these developments, however, some problems still exist in the area of document comprehension. One problem is to maintain consistency of meaning throughout lengthy documents, while another issue concerns the efficient integration of multimodal information. The problem of minimizing computational complexity is also not yet satisfactorily addressed by current models, as well as the problem of achieving generalization across various kinds of documents.

# CHAPTER-3
# PROPOSED METHODOLOGY

This chapter provides an explanation of the methodology chosen for the development of the document understanding model. Information about the datasets selected for classifying and summarizing documents is provided, together with their properties. This chapter also sheds light on the preprocessing performed before processing both visual and textual input data, including transformations and splitting documents into chunks. Such preprocessing guarantees the compatibility of both visual and textual inputs with the use of transformer models.

Additionally, this chapter provides a detailed description of the feature extraction and modeling approaches used in the proposed model. The feature extraction process uses layout-aware features that are extracted from the documents using the Document Image Transformer (DiT). As for the summarization of documents, the BART and Longformer Encoder-Decoder (LED) approaches are applied to generate the summary based on the segmented text. Moreover, there are approaches that make it possible to identify the relationships between various document segments.

Furthermore, the chapter also explores the methods employed to extract the most relevant data from the processed information. These include similarity calculation and interaction analysis, which help extract important sections from the documents, whereas Maximum Marginal Relevance helps ensure that the extracted data is both relevant and redundant. This chapter gives a detailed account of all the processes involved in the entire pipeline, including pre-processing, feature extraction, modeling, and information selection.

## 3.1 SYSTEM ARCHITECTURE

The architecture of the system under study ensures a logical way for carrying out classification and summarizing long documents using the developed approach. The process involves input documents, which first undergo preprocessing steps like image transformation and document chunking. To perform classification, a Vision Transformer-based Document Image Transformer (DiT) model is used, which allows extracting features and making document embeddings for accurate classification. To carry out summarizing, the initial document is chunked, and a transformer-based approach is used to encode it and use some similarity measures for recognizing crucial parts of it. Such techniques as Maximum Marginal Relevance (MMR) are used to avoid any redundancy, while BART and LED are chosen to make summaries.

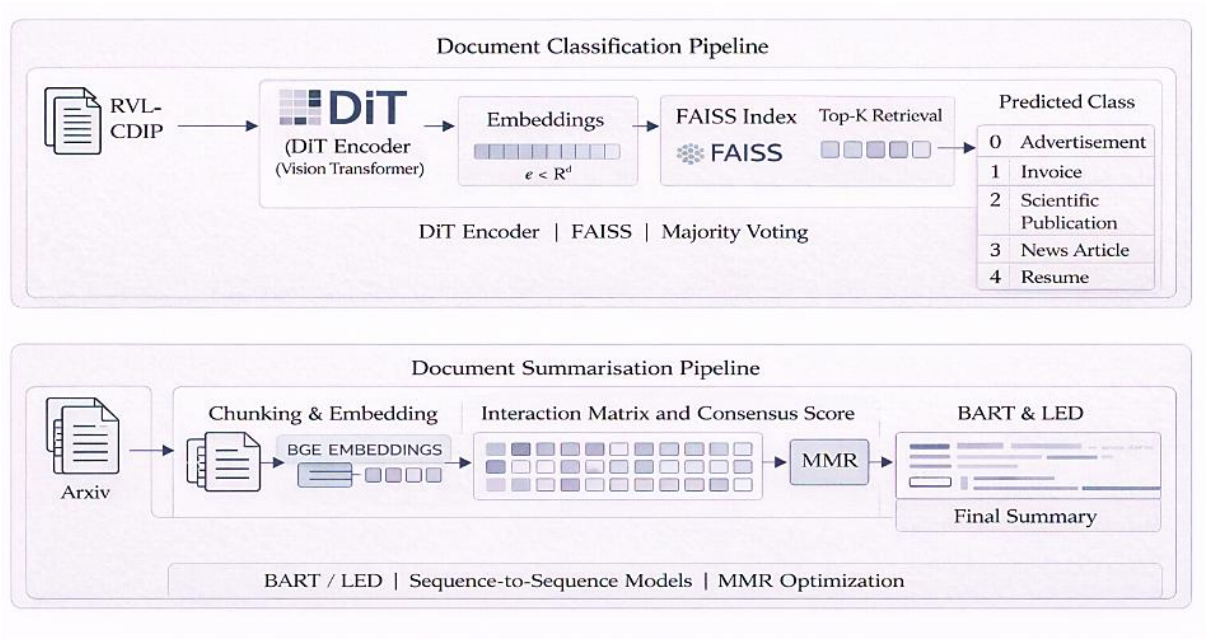


***Fig 3.1: Architecture Diagram***

The architecture shown in Figure 3.1 below, comprising two major pipelines, namely document classification and document summarization. In the document classification pipeline, the input documents are extracted from the RVL-CDIP dataset and encoded with DiT (Vision Transformer), which produces embeddings. The produced embeddings are indexed with the help of FAISS for similarity search and Top-K retrieval, with the final result being obtained with majority voting on the predicted class, which could be any of Advertisement, Invoice, Scientific Publication, News Article, and Resume. On the other hand, in the summarization pipeline, the input documents, say from Arxiv, are chunked and embedded using BGE embeddings. An interaction matrix and consensus scoring is then done before employing Maximal Marginal Relevance.

### 3.1.1 Document Classification

The purpose of the document classification model is to determine the class of the document in question using the information about its layout and structure. As illustrated in Fig. 3.1, the architecture of the proposed system includes the use of a vision transformer-based Document Image Transformer (DiT), which can obtain representations of documents that are aware of their layouts. The process of document classification in the proposed system involves patch embeddings with positional encoding, transformation-based feature extraction, and global document representation.

**Patch Embedding with Positional Encoding:**

For transforming document images using transformers, the document image must first be split into patches of equal size. These patches will then correspond to small portions of the

document image. This makes it possible for the model to treat an image in a similar way that words are processed in a sentence. After flattening the patches into one-dimensional vectors, the patches are embedded into a dense space by applying a linear transformation on the vector space, as shown below:

$$Z_i = W_e x_i \qquad Eq - (1)$$

where $x_i$ denotes the flattened representation of the i-th image patch, $W_e$ is the learnable projection matrix, and $z_i$ is the resulting patch embedding. These embeddings act as the input tokens to the transformer model.

In order to retain the spatial relationship between elements within the document, a positional embedding is included into the patch embedding. Transformers are unaware of the order or the position of the tokens within their inputs, and so the use of positional encoding allows the model to know the relative or absolute positioning of the patches within the document. This is highly significant when working with document images, since layout components like the position of headings, paragraphs, and tables are essential parts of comprehending the content of the document.

$$Z = \{z_1, z_2, \ldots\ldots, z_N\} + E_{Pos} \qquad Eq - (2)$$

where Z represents the sequence of embedded patches and E{pos} denotes the positional embeddings that encode spatial information.

**Transformer Encoder**

The chain of patch embeddings goes through several layers of the transformer encoder. In every consecutive layer, the representation of the document is updated as a combination of its visual layout structure and relations within the different regions of the document. The encoder plays an important role in the suggested system for document understanding that is based on the Visual Transformer architecture called Document Image Transformer (DiT). The encoder allows recognizing the structure of documents with the help of headings, paragraphs, tables, etc. As opposed to convolutional architectures, the transformer encoder can capture global dependencies since all patches interact with each other.

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \qquad (Eq - 3)$$

where Q, K, and V represent the query, key, and value matrices, and dk represents the dimension of the key vectors.

With self-attention, each patch is made to attend to other meaningful patches in the image of the document, giving greater weight to those regions that are either structurally or

semantically important, like titles and headings.

**Document Embedding Generation:**

The document embedding generation phase converts the output of the transformer encoder into an optimal representation of the whole document. As the self-attention mechanism and feature extraction are applied several times in the document encoding stage, each patch embedding will gain awareness of its surrounding context, which involves not only visual but also semantic information about the document. In our solution, a specific [CLS] token helps to merge all patch embeddings into a single representation of the whole document image. The resulting global representation carries important information related to the structure of the document, such as headings, sections, and other aspects that allow classifying documents into particular categories.

$$E = f_{DiT}(I') \qquad (Eq - 4)$$

where $I'$ represents the processed document image, fDiT denotes the Document Image Transformer encoder, and E is the extracted document embedding.

The created document embedding acts as a feature vector at a higher level that captures the input image $I'$. This embedding is later used for other operations like similarity search and classification. In your architecture, this embedding is indexed through FAISS in order to perform similarity searches, whereby documents having similar structural and semantic attributes are placed in the same cluster. This enhances the performance of the classifier by taking advantage of learned representations instead of just depending on raw textual or visual attributes.

### 3.1.2 Document Summarisation

The purpose of the document summarization module is to generate a summary for the given documents containing all the relevant and important information in the document. As illustrated in Fig. 1, the document summarization architecture first encodes the document chunks using the transformer encoder for understanding the contextual relationships among various document segments. Finally, the system understands the relationship among various document segments and selects the most relevant and non-repetitive information for the generation of document summaries.

**Chunk Encoding:**

Chunk encoding is a very important stage in the process of document summarization because the long documents are split into smaller units called 'chunks' to handle the input limit in transformer models. The 'chunk' in the document refers to a smaller segment of the documents which keeps all contextual information within it. In this document

summarization task, after performing the preprocessing and chunking, each chunk will be fed to a transformer encoder like BART or Longformer Encoder-Decoder (LED).

$$e_i = f_{enc}(c_i) \qquad (Eq - 5)$$

For each chunk, a contextual embedding is created in the encoder layer, such that, $c_i$ represents the input chunk while $e_i$ is its vector representation. The chunk representations include semantic features and contextual relations in each chunk; therefore, they can be analyzed in further stages. In the designed model, the obtained chunk embeddings will be used in the calculation of interactions and importance measures of all segments. Consequently, this stage will make sure that the process of summarization considers contextual aspects of each chunk and relationship among chunks.

**Interaction Matrix and consensus score:**

To determine the relationships among different document chunks semantically, an interaction matrix is employed to show how two chunks relate to each other semantically in the document. By using the transformer encoder on all document chunks, the system compares embeddings of each chunk to each other. The comparison of the vectors makes it possible to evaluate the similarity between any section and other sections in the document. Through this stage, it is possible to see which information pieces are similar to one another in the document.

The consensus score of each chunk is generated from the interaction matrix to measure its significance in the entire document. The significance of the chunks refers to their interaction strength with other chunks, showing whether or not they contain important information that can be shared among others. The chunks with higher significance scores are regarded as more informative and more relevant to the content of the document. In this work, the more significant chunks are selected to form a summary of the document.

**Key sentence selection using MMR and Summary Generation**

After scoring based on the consensus, key sentences that will be most relevant without redundancies will be selected. This will be achieved by use of a concept called Maximum Marginal Relevance.

$$MMR = arg \max_{s_i \epsilon D \backslash s} \left[ \lambda Sim(s_i, q) - (1 - \lambda) \max_{s_j} Sim(S_i, S_j) \right] \qquad (Eq - 6)$$

where Sim(Si,Q) represents the similarity between sentence Si and the document, Sim(Si,Sj) represents the similarity between sentence Si and other sentences selected, and λ represents the balancing factor between relevance and redundancy. Lastly, the key sentences selected are used to generate the final summary through the transformer model. In this model, two

transformers are utilized for generating long documents. These transformers include BART and LED.

## 3.2 DESIGN METHODOLOGY

In this section, we will discuss the design method of the proposed system, which emphasizes designing an integrated system for document classification and summarization of long documents. It includes the selection of appropriate datasets and the use of preprocessing methods, including resizing images, normalizing them, and dividing documents into smaller chunks before feeding them to transformers. This system employs Vision Transformers-based Document Image Transformers (DiT) to extract layout-sensitive features for classification purposes. On the other hand, the BART and Longformer Encoder-Decoder (LED) models are utilized to generate summaries from long documents.

### 3.2.1. System Design

The methodology is intended to create a hybrid document understanding system where both document classification and document summarization are merged in one framework. Such an approach is made possible due to the use of vision transformers and language transformers for the processing of visual layout information along with text information available in documents. As a result, the proposed framework will be capable of analyzing multi-page documents that have tables, headings, figures, paragraphs, etc.

In order to achieve these purposes, the methodology utilizes a modular approach to building the system, including several stages such as data acquisition, data preprocessing, feature extraction, classification, and summarization. Every stage is designed to do its job and be interacted with other modules without any difficulties. The purpose of the classification module is to classify the document, whereas the summarization module analyzes it and creates a summary.

### 3.2.2. Dataset Selection and Preparation

The efficiency of the suggested model largely relies on the choice of appropriate datasets for both structural and text diversification. In case of document classification, the RVL-CDIP dataset is used, as it provides a vast amount of diverse document types, including advertisements, invoices, scientific documents, news papers, and CVs. With the help of this dataset, the model learns discriminative visual properties of documents based on the structure.

In the case of long-document summarization, the ArXiv dataset is utilized, where each long scientific document is complemented by its abstract. Such documents allow for testing the ability of the summarizer in the process of generating high-quality abstracts, as

there is enough structured information within them. The model will be able to establish the relation between full documents and the abstracts.Data preparation for the dataset includes organizing the data, making sure that there is an even distribution of classes, and formatting it to be ready for use in the training process.

### 3.2.3. Data Preprocessing and Transformation:

The data preprocessing phase is important in the design method, where raw input data is formatted into the required format for feeding the model. In the case of document classification tasks, each document will be represented in the form of an image where layout-based features can be extracted. The images are standardized to the same resolution, enabling the model to extract structure from the images regardless of the document length differences.In the document summarization task, preprocessing involves breaking down longer documents into smaller pieces. The transformer model cannot process longer sequences of data; hence, this preprocessing enables the input to be broken into small sections without loss of context. The text is then tokenized to enable it to feed the transformers. This preprocessing process enhances noise reduction and model efficiency.

### 3.2.4. Patch Tokenization and Feature Extraction

In order to use transformer-based models for processing images of documents, the input images are split into small patches of fixed dimension. The patches act as representations of a specific area on the image and are converted into a vector format. This way, patches can be embedded into a highly dimensional space to extract important visual information.

Positional encoding is introduced to maintain the spatial order of patches, so that the network can grasp the spatial relationship of different components in the document. It is especially crucial for document understanding problems because layout information has a substantial impact on classifying different documents.Patch embedding along with positional encoding helps the network capture both local and global information of the document, making a solid base for document classification.

### 3.2.5. Document Classification Module Design

The classification component is implemented by using the Vision Transformer architecture that consists of the Document Image Transformer (DiT). The model applies transformations to a sequence of the patches embedding through several encoder layers, where the self-attention operations can detect relationships between different parts of the image. As a result of training, the model will be able to detect headers, different sections of text, alignment, and formatting style. The document image will be encoded with the help of a specific classification token as a global feature vector of the entire document.

To optimize the classification component, the model uses a similarity search based on Faiss indexing. Thus, the document image will be compared with other embeddings, and the model will be able to select an optimal label for it.

### 3.2.6. Document Summarization Module Design

The task of generating a succinct and relevant summary from large pieces of text in this summarization module is ensured through the inclusion of contextual information within the summary. In the beginning, the text is broken down into chunks, which are then processed into embeddings using transformer architectures like BART and LED encoder-decoder models. These chunks are converted into contextual embeddings to extract the semantics of the sentences. After this, interactions between these embeddings are examined via an interaction function to understand their significance, followed by a consensus score to determine the relevancy of the sentences in the text.Maximum Marginal Relevance (MMR) is used to select those sentences from the text that are significant but not repetitive in nature. These sentences then go through a decoder to provide a meaningful output summary.

## 3.3 DESCRIPTION OF DATASET

**Python code instruction dataset:**

The dataset code instruction dataset consists of 18,612 entries and 4 columns:

Instruction - A text-based instruction describing a programming-related task.

Example- Write a python program to sort the numbers.

Output - The expected output, typically a code snippet or result of executing the instruction.

**Dataset Link:**

https://www.kaggle.com/datasets/thedevastator/python-code-instruction-dataset

**Code Alpaca Dataset:**

The Code Alpaca dataset contains 20,000 entries and 3 columns(SQL, Python, C++)

Instruction – A natural language prompt describing a coding task.

Example- create a table employee in SQL.

output – The expected code or explanation generated for the instruction.

**To load the dataset:**

```
from datasets import load_dataset
ds = load_dataset("sahil2801/CodeAlpaca-20k")
```

**Document Classification and summarisation:**

Two datasets are utilized in this research to assess the effectiveness of the proposed framework in document classification and document summarization tasks.

**Document Classification**

The dataset comprises documents with various layout and structural patterns. As shown in Table 3.7, the dataset contains five document classes with approximately 600 samples per class. For document classification, the RVL-CDIP dataset is utilized. RVL-CDIP is a benchmark dataset commonly utilized to assess document image classification models. It comprises a large number of images of documents with various categories, including advertisements, invoices, scientific documents, news articles, and resumes.

***Table 3.7: Class-wise Distribution of RVL-CDIP Dataset***

| **Document Class** | **Number of Samples** |
|---|---|
| Advertisement | 599 |
| Invoice | 599 |
| Scientific Publication | 597 |
| News Article | 601 |
| Resume | 601 |
| **Total** | **2997** |

**Document Summarization:**

In the case of document summarization, a database named ArXiv is used. This dataset comprises scientific documents which generally contain a large amount of text along with certain structuring in the form of sections, figures, and tables.

A sample of document summarization database contains two parts:

**Article:** The complete text of document comprising sections and descriptions.

**Abstract:** Summary of the text containing important aspects of document.

### 3.3.1 Data Preprocessing:

Before applying the proposed models, a set of preprocessing steps is carried out to prepare the datasets for document classification and summarization. These steps ensure that the input data is properly formatted and is compatible with the proposed models based on transformers.

**Image Preprocessing for Document Classification:**

For the document classification task, each document is converted to an image format to enable the extraction of layout features using the Document Image Transformer (DiT)

model. For this purpose, it is essential to transform each document image to a fixed resolution to be able to use the vision-based transformers.

Let the input document image be represented as I. The preprocessing step is defined as:

$$I' = Normalize\big(Resize(I)\big) \qquad Eq - (7)$$

Where I' denotes the processed image that is provided as input to the DiT model.

**Patch Tokenization:**

The division of the input image into smaller patches is important as transformers require sequential data for their processing capabilities. To accommodate this requirement, the image needs to be split into patches that will represent the two-dimensional data in a sequence. This approach is necessary since the input data will be processed according to its visual content instead of its spatial characteristics.

Patches contain small regions of the document and may include information about texts, headings, and other elements of layout. All obtained patches are then transformed into vectors. Thus, each patch becomes a vector of features, making it easier to convert visual data into sequences and work with them in the context of the transformer model. It should be noted that after being flattened, the patches become tokens that function similarly to words in language.

The second important step in the process is patch embedding in which each patch is projected into a high-dimensional embedding space. As a result, patches will be enriched with meaningful features and become easily interpretable by the model. However, since transformers do not possess intrinsic positional information, such data will need to be added to embeddings.

The last batch of embedded patches serves as the input to the encoder of the transformer model. Self-attention allows the model to establish relationships between various patches to identify dependencies between the patches. These dependencies are necessary for understanding the structure of the document and improving the accuracy of classification tasks.

If the image resolution is H*W and the patch size is P*P, the total number of patches N can be computed as

$$N = \frac{H * W}{P^2} \qquad Eq - (8)$$

Each patch is then flattened and transformed into a vector representation that serves as input to the transformer encoder.

**Document Chunking for Summarization**

Regarding the problem of document summarization, it is vital to conduct pre-processing to deal efficiently with documents that have much textual data. The transformer architecture has a constraint with regard to the length of an input sequence for processing. For this reason, it is not possible to provide an entire document to the model at once since this can affect its performance adversely. Consequently, it becomes necessary to divide the document into small parts referred to as chunks.

Every chunk is a part of a document characterized by the continuity of its content and containing meaningful information about the document. It is important to take into account that the document should be split into chunks in such a way that does not break the context, and therefore sentences should not be arbitrarily split into different parts.

Chunking further ensures that parallel or serial processing of lengthy documents becomes possible by analyzing each part separately but maintaining the entire document's structural integrity. As a result of breaking down into smaller parts, computational overhead is minimized, and memory is effectively utilized, thus enabling efficient processing of large documents.

Chunking further enhances representation learning as each part can be embedded into meaningful embeddings based on local context. The learned embeddings are then compared or merged, which helps determine the most important portions of the document. This is an essential process since it ensures that the resultant summary contains all important information across various parts of the document.

Assume that the input document is given by D. The chunking of documents is defined as:

$$D = \{c_1, c_2, \dots \dots, c_n\} \qquad Eq - (9)$$

This helps the model to process the long documents efficiently, and the context relationships between the sections of the documents are maintained.

## 3.4 DESCRIPTION OF ALGORITHMS

This study employs transformer-based encoder-decoder architectures – CodeT5 and BART – for automatic code generation based on natural language prompts.

**CodeT5:**

CodeT5 refers to a transformer-based encoder-decoder architecture created specifically for coding tasks via the development of the Text-to-Text Transfer Transformer (T5).The algorithm of codeT5 is shown in Table 3.1. CodeT5 is trained to comprehend both natural and coding languages. Hence, it is effective at bridging the divide between humans' instructions and the codes. The model undergoes pre-training on large code corpora from GitHub, in which it learns

coding conventions, syntactic structure, and semantics through objectives such as span masking and token classification. CodeT5 can carry out coding tasks such as generation, summarization, and translation accurately and efficiently while maintaining significant aspects like variable names and code structure.In this particular case, the use of CodeT5 is adopted for generating code from natural language prompts provided by the users. The fine-tuning of this particular model can be achieved by using related training datasets containing both the input prompt and the associated code snippet. At runtime, the input prompt undergoes encoding to understand its context, whereas the decoding stage helps in generating the corresponding code sequentially. Such an approach not only helps in the creation of valid syntax but also enables it to capture the context of code generation.

***Table 3.1: CodeT5 Algorithm***

Algorithm: CodeT5 for Code Generation

Input: Natural language description of code

Output: Generated code snippet

Method:

1. Load the CodeT5 model and Tokenizer (Salesforce/codet5-base).

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
tokenizer = AutoTokenizer.from_pretrained("Salesforce/codet5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("Salesforce/codet5-small
```

2. Preprocess Data
3. Tokenize the input text using the CodeT5 tokenizer with padding and truncation.
4. Define the training arguments using train() function.
5. For epoch in range(num_epochs):

For each batch of data:

Pass tokens, attention masks, and labels to the model.

Compute the logits (outputs) and loss.

Backpropagate the loss to update weights.

8. Calculate metrics (e.g., BLEU score, code similarity) on the validation set.

**Bidirectional and Auto-Regressive Transformers(BART):**

Bidirectional and Auto-Regressive Transformers, or simply BART, is a state-of-the-art sequence-to-sequence model created by Meta AI. The BART algorithm for code generation is shown in Table 3.2. In essence, BART consists of a bidirectional transformer encoder that

resembles BERT in terms of architecture and an auto-regressive decoder, similar to GPT. Such a combination allows BART to take advantage of the bidirectionality and understanding of the full context from the encoder and generation capabilities of the decoder, thus being able to generate text while understanding the input. BART can be pre-trained as a denoising auto-encoder by reconstructing the corrupted text and learning deep contextual representation. As a result of such an architecture, BART is suitable for performing multiple text generation tasks, including summarization, machine translation, and question answering.

The current study explores BART as a means of performing code generation. To do so, the task was re-interpreted as structured text generation. BART has been fine-tuned to work with pairs of prompts that consist of natural language descriptions of programming concepts and corresponding pieces of code. Then, during inference, the encoder part of the model would receive an input prompt and capture its meaning. The generated output code would be obtained through autoregression performed by the decoder.

***Table 3.2: BART Algorithm***

| |
|---|
| Algorithm: BART for Code Generation |
| Input: Natural language description of code |
| Output: Generated code snippet |
| Method: |
| 1. Load the Pretrained BART Model and Tokenizer(facebook/bart-large). |
| 2. Preprocess Data |
| 3. Create input IDs and attention masks for both the encoder (input) and decoder (labels). |
| 4. Set parameters such as number of epochs, batch size, learning rate, evaluation strategy, and checkpoint saving. |
| 5. Training Loop: |
| a. For each batch of training data: |
| i. Pass input IDs and attention masks to the BART model |
| ii. Compute the output logits and calculate loss using the provided labels |
| iii. Backpropagate loss to update model weights |
| 6. Calculate metrics (e.g., BLEU score, code similarity) on the validation set. |

**LLaMA-3.3-70B Versatile:**

LLaMA-3.3-70B Versatile is a transformer-based large language model created by Meta AI, which aims at developing abilities of reasoning, high-level code understanding, and creating structured texts. During processing of the input sequence, the model uses multiple layers of

self-attention to be able to discover dependencies between different parts of the code such as variables, functions, and other components. Due to the large number of parameters, the model learns dependencies in the data and acquires an understanding of programming language syntax and logic. Such a model is very helpful for any tasks that require analysis and understanding of code.

***Table 3.3: llama-3.3-70b-verstaile***

**Algorithm:** llama-3.3-70b-verstaile

**Input:** Code snippet

**Output:** Bug report, corrected code, and test cases

**Method:**

1. **Load Models** – Load the LLaMA-3.3-70B Versatile along with their tokenizers.
2. **Prepare Prompts** – Create simple instructions for each task: bug detection, fix suggestion, corrected code, report, and test cases.
3. **Preprocess & Tokenize** – Clean the input code and convert it into tokens using the respective tokenizers.

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^t}{\sqrt{d_k}}\right)V \qquad \text{[Eq-1]}$$

4. **Pass Through LangGraph Nodes** –
   Bug Detection Node → find errors
   Fix Suggestion Node → suggest corrections
   Corrected Code Node → generate final fixed code
   Report Node → explain issues
   Test Case Node → generate tests
5. **Model Inference** – Each node sends the tokenized input to the LLM, which processes it and returns decoded text outputs.
6. **Decode Output** – Convert the predicted tokens back into readable text or code.

**Finalize Results** – Collect the bug list, corrected code, explanation report, and test cases as the final output.

The chosen application for the project involves using the LLaMA-3.3-70B Versatile language model for detecting and fixing bugs and performing code analysis. In order to do that, the model analyzes the program and highlights all problems related to logic or syntax or other issues. Moreover, the system is able to compare the desired program flow and output with what it

actually looks like and determine places that can cause some difficulties. Furthermore, the model can offer some fixes and suggestions in order to make the program work properly. It relies on self-attention to establish relationships in code contexts that make it ideal for detecting syntax and semantics errors. For bug detection, LLaMA can learn the patterns in the source code to identify bugs through its training on vast amounts of code data. The algorithm used in bug detection using LLaMA is shown in Table 3.3

**Qwen3-Next-80B-A3B-Instruct**

Qwen3-Next-80B is a large-language transformer-based model designed and developed by Alibaba Cloud. The algorithm for qwen3-Next-80B-A3B-Instruct is shown in table 3.4.

***Table 3.4: Qwen3-Next-80B-A3B-Instruct***

**Algorithm:** Qwen3-Next-80B-A3B-Instruct

**Input:** Code snippet

**Output:** Bug report, corrected code, and test cases

**Method:**

1. **Tokenization & Encoding** – Source code is tokenized using the Qwen tokenizer; embeddings and positional encodings are added.
2. **Transformer Processing** – 80-layer decoder applies multi-head self-attention.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \qquad [Eq - 1]$$

Followed by feed-forward networks to learn code patterns.

3. **Prompt Conditioning** – System and human prompts instruct the model to perform debugging, fixing, and test generation.
4. **Decoding** – Model predicts tokens using causal decoding:

$$P(y_t \mid y_{<t}, x) = \text{softmax}(Wh_t) \qquad [Eq - 2]$$

To generate reports, corrected code, and test cases.

**Output Assembly** – Predicted tokens are detokenized into readable text/code to produce the debugging report, fixed source code, and test cases.

It is specifically trained for instruction-following, multilingual reasoning, and code understanding tasks. Through deep self-attention mechanism, this model is able to analyze input prompts step-by-step to understand not only natural language instructions but also programming logic. The main focus during Qwen3-Next-80B's training is made on perfect

alignment of generated output with users' instruction. This helps the model to provide structured and contextually-relevant answers. In addition, due to multilingual ability, it is able to understand prompts and code-related queries in various languages.

The current project uses Qwen3-Next-80B to detect bugs and analyze code. To do that, the model interprets input code and finds inconsistencies and potential issues like naming problems, logical flaws, and mismatches between actual function and the one described in the prompt. The model evaluates the program flow and understands if it was implemented according to users' intentions and needs. As a result, Qwen3-Next-80B can generate structured responses, including description of errors, debugging process steps, and corrected code.

**DiT-Base Model**

DiT-Base is a transformer-based deep learning model specifically designed for document image classification. The algorithm for DiT is shown in Table 3.5.

***Table 3.5: DiT(Document Image Transformer)***

| |
|---|
| **Algorithm:** DiT-Base for Document Classification |
| **Input:** Document(word,pdf) |
| **Output:** Predicted Class label |
| **1. Image Preprocessing** The input document image is resized to a fixed resolution (e.g., 224×224) and normalized so that it can be properly processed by the transformer model. |
| **2**. **Patch Tokenisation** The image is divided into small fixed-size patches, converting the 2D image into a sequence of smaller image regions. |
| 1. **Patch Embedding with Positional Encoding** Each patch is flattened and projected into a dense vector representation (embedding) using a learnable linear layer, converting image patches into tokens. Positional embeddings are added to each patch embedding to preserve spatial layout information of the document. |
| **5**. **Transformer Encoder** The embedded patches pass through multiple transformer layers where self-attention captures relationships between different document regions. The [CLS] token output is used as the global representation of the entire document image. |
| **6**. **Classification** The global representation is passed through a linear layer followed by Softmax to predict the document class. |

It adapts the vision transformer architecture to effectively process document images by dividing them into smaller patches and converting them into embedding representations. These embeddings are passed through multiple layers of a transformer encoder, where self-attention mechanisms capture both local and global relationships across different regions of the document. This enables the model to understand complex layout structures such as headings,

paragraphs, tables, and visual patterns, which are essential for distinguishing between different types of documents.

In this project, DiT-Base is used as the core model for document classification by leveraging its ability to extract layout-aware features from document images. The model learns structural patterns unique to each document category, such as the format of invoices, the layout of resumes, or the structure of scientific publications. The final output representation generated by the model is used to classify documents into predefined categories like advertisement, invoice, scientific publication, news article, and resume. This approach improves classification accuracy by focusing not only on textual content but also on visual and spatial characteristics of the document.

**Document Summarization (BART and LED):**

The summarization process of documents forms an essential part of NLP tasks, as it enables the generation of concise summaries while preserving the core meaning and important information of the original text.

***Table 3.6: Document Summarization Algorithm***

| |
|---|
| **Algorithm: Document Summarization** |
| **Input:** Documents |
| **Output:** Summary of the document |
| **1. Document Chunking**: The long input document is divided into smaller chunks to efficiently handle large text inputs and avoid memory limitations. |
| **2. Chunk Encoding**: Each chunk is passed through the transformer encoder (BART/LED) to generate contextual embeddings that capture the meaning of the text. |
| **3. Interaction Matrix Creation**: Similarity scores are computed between chunks to capture relationships and understand how different parts of the document are related. |
| **4. Consensus Scoring**: Each chunk is assigned a consensus score based on its overall interaction with other chunks, helping to identify the most important content. |
| **5. Key Sentence Selection (MMR Optimization)**: Important and non-redundant sentences are selected using MMR, ensuring high relevance while minimizing duplication in the summary. |
| **6. Summary Generation**: The selected key content is passed to the decoder, which generates the final coherent summary of the document. |

This is particularly useful when dealing with large volumes of data such as research articles,

reports, and news documents, where extracting key insights quickly is crucial. Summarization techniques can be broadly categorized into extractive and abstractive methods, with transformer-based approaches predominantly supporting abstractive summarization by generating new, coherent sentences.

Transformer models have proven to be highly effective in this domain due to their ability to capture contextual relationships and semantic dependencies within the text. By leveraging attention mechanisms, these models can focus on the most relevant parts of a document and generate meaningful summaries that maintain logical flow and coherence. Table 3.6 outlines the algorithm used in the summarization process, detailing steps such as preprocessing, tokenization, encoding, and summary generation. This structured approach ensures that the model efficiently processes input text and produces high-quality summaries.

Furthermore, the integration of techniques such as attention-based scoring and redundancy reduction enhances the overall quality of the summaries. These methods help in selecting the most informative content while avoiding repetition, thereby improving readability and effectiveness. As a result, transformer-based summarization models play a vital role in modern document understanding systems, offering scalable and accurate solutions for handling large textual datasets.

The latest models being used for abstractive summarization include BART and LED which are transformer-based models.

BART is an encoder-decoder model which utilizes the benefits of bidirectional encoders such as BERT and autoregressive decoders like GPT. Such a model enables BART to comprehend the full context of the input text while producing accurate summaries.

This model is based on the denoising autoencoder methodology, where the model is trained to regenerate the original text from noisy texts. Such a technique makes the model capable of producing summaries which are fluent, grammatical, and semantic. Because of the strong context-based nature of the model, BART works very efficiently for short to medium-length texts. It is particularly built to cater to lengthy texts that surpass the normal maximum number of tokens that other transformers can take. It has a sparse attention mechanism, whereby it cuts down the computational resources used by the transformer and allows it to attend to a larger number of tokens.

LED has both local and global attention. Local attention allows it to pay attention to tokens within the vicinity, while global attention makes it possible for the transformer to focus on important tokens in the entire text. As such, the LED is quite effective when dealing with long documents.

The proposed system offers an all-in-one system that combines the tasks of classification and summarization in one comprehensive framework. It utilizes preprocessing techniques, transformer architecture, and advanced mechanisms of information selection to facilitate effective and efficient understanding of the document.

Some important benefits of the proposed system are:

- It will be scalable enough to handle short and long documents alike.
- It will enable generation of more meaningful summaries by gaining better context comprehension from the text.
- It will be able to minimize redundancy using optimization such as MMR.
- In essence, the incorporation of BART and LED allows us to create a highly efficient summarizer with minimal redundancy.

In summary, the method proposed here offers a comprehensive approach to document understanding by integrating classification and summarization processes within a single framework. Through the use of preprocessing techniques, advanced transformer-based models, and an effective information selection mechanism, the system ensures efficient and accurate handling of documents. The classification module successfully identifies document types based on both textual and layout features, while the summarization module generates concise and meaningful summaries that preserve the core information.

Furthermore, the integration of these components enables a seamless workflow, where the output of the classification stage supports improved summarization performance. The use of layout-aware models and attention mechanisms allows the system to capture both structural and semantic aspects of documents, making it suitable for handling diverse document formats such as reports, articles, and invoices. This combined approach enhances the overall reliability and applicability of the framework in real-world scenarios.

Additionally, the incorporation of techniques like Maximal Marginal Relevance (MMR) ensures that the generated summaries are not only relevant but also non-redundant, improving information quality. The framework also demonstrates scalability and adaptability, as it can be extended to other document-related tasks such as information extraction and question answering. Overall, the proposed method provides an efficient, accurate, and practical solution for intelligent document understanding systems.

Finally, the proposed framework also demonstrates strong potential for future enhancements and real-world deployment. By incorporating more advanced pretraining techniques, larger and more diverse datasets, and domain-specific fine-tuning, the system can be further improved to handle complex and specialized documents. Additionally, integrating interactive features such as user feedback mechanisms and explainable AI components can enhance transparency and usability. This makes the framework not only a robust solution for current document understanding tasks but also a scalable foundation for developing more intelligent and adaptive document processing systems in the future.

# CHAPTER – 4
# RESULTS AND OBSERVATION

This section discusses the results obtained from evaluating the proposed document understanding framework for both document classification and document summarization tasks. The classification component is assessed using the RVL-CDIP dataset, while the summarization module is evaluated on the arXiv dataset. These datasets provide diverse and real-world document structures, enabling a comprehensive analysis of the system's performance across different document types and complexities.

For document classification, the results demonstrate that the model effectively identifies and categorizes documents based on their visual layout and textual features. High accuracy and strong performance metrics indicate that the model successfully learns discriminative features across multiple document classes. The confusion matrix further supports these findings by showing minimal misclassification, with only slight confusion observed in visually similar document categories.

In the case of document summarization, the framework generates concise and meaningful summaries that retain the essential information from lengthy documents. The evaluation using standard metrics highlights the model's ability to maintain semantic relevance, coherence, and structural consistency. Comparative analysis between different models shows that the system performs particularly well for long documents, ensuring better context preservation and reduced information loss.

Overall, the results validate the effectiveness of the proposed framework in handling both classification and summarization tasks efficiently. The use of diverse datasets and multiple evaluation metrics ensures a robust assessment, demonstrating that the system is capable of delivering accurate and high-quality outputs in real-world document understanding scenarios.

## 4.1 DESCRIPTION OF RESULTS

This section provides an overview of the outcomes obtained from the implementation of the proposed system. It highlights the performance of the framework across different components, including document classification, summarization, code generation, and bug detection. The results are analyzed using various evaluation metrics and visual representations to assess the accuracy, efficiency, and reliability of the models.

The section also discusses key observations and comparisons between different models, emphasizing their strengths and limitations. Through this analysis, the effectiveness of the

proposed approach is validated, demonstrating its capability to handle real-world scenarios and deliver meaningful results.

#### 4.1.1. Code Generation

The code generation task focuses on evaluating the effectiveness of transformer-based models in generating syntactically and semantically correct code from natural language instructions. In this experiment, models such as CodeT5 and BART were utilized to generate code for given problem statements. These models leverage attention mechanisms to understand contextual relationships within input text, enabling them to translate human language into structured programming logic.

The generated outputs were assessed using standard evaluation metrics such as Exact Match (EM), BLEU, and CodeBLEU. Exact Match measures whether the generated code exactly matches the reference solution, while BLEU evaluates n-gram similarity between generated and target code. CodeBLEU provides a more comprehensive evaluation by incorporating syntactic structure, data flow, and semantic correctness, making it particularly suitable for code generation tasks.

The results demonstrate that transformer models are capable of producing coherent and functionally relevant code. CodeT5 showed superior performance in preserving code structure and handling programming-specific syntax, whereas BART generated grammatically consistent outputs but occasionally lacked logical precision. However, challenges such as ambiguity in input instructions and maintaining long-range dependencies were observed, indicating the need for further optimization and fine-tuning to improve overall performance.

**CodeT5 Model:**

The results of the generated code through the proposed code generation model were assessed using BLEU, ROUGE-1, ROUGE-2, ROUGE-L, METEOR, and BERTScore (F1). The evaluation outcomes are illustrated in Fig. 4.1. The ROUGE-L score of 0.4425 indicates that the model effectively preserves sequence-level structure in the generated code, ensuring logical continuity. Additionally, the METEOR score of 0.4208 reflects a balanced performance in terms of precision and recall, highlighting the model's ability to produce relevant and moderately accurate outputs.

Furthermore, the highest performance was observed in the BERTScore (F1), which achieved a value of 0.9021. This high score demonstrates strong semantic similarity between the generated code and the reference code, indicating that the model successfully captures the underlying intent of the input instructions. Compared to traditional n-gram based metrics,

BERTScore provides deeper contextual evaluation, reinforcing the effectiveness of transformer-based architectures in understanding complex programming requirements.

Overall, the evaluation results suggest that the proposed model performs well across both syntactic and semantic dimensions. While metrics such as BLEU and ROUGE confirm reasonable structural alignment, the strong BERTScore highlights superior contextual understanding. However, there is still scope for improvement in exact code matching and logical precision, particularly for complex or multi-step problem statements, which can be addressed through further fine-tuning and dataset enhancement.

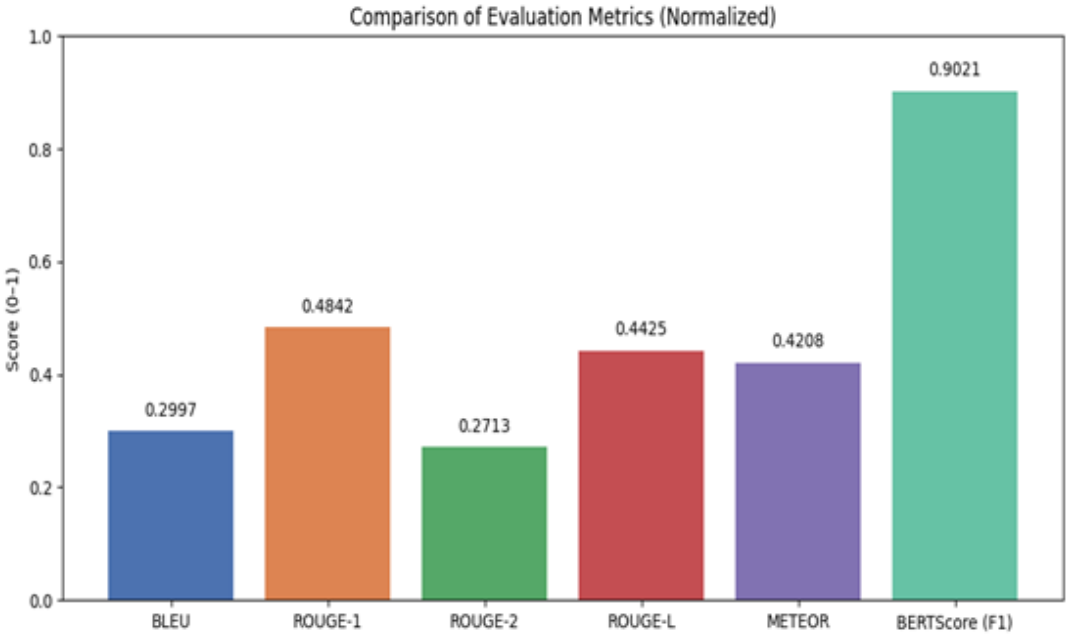


***Fig 4.1: Evaluation of CodeT5***

The ROUGE-L score of 0.4425 proves that the proposed model is capable of maintaining sequence-level information in the produced code. Moreover, the METEOR score of 0.4208 shows that the performance is balanced in terms of precision and recall. It is worth noting that the highest value was obtained for the BERTScore (F1), which amounted to 0.9021.

**BART Model:**

Furthermore, the BERTScore (F1) obtained for the BART model demonstrates strong semantic alignment between the generated code and the reference implementation. This indicates that, beyond surface-level similarity, the model is capable of capturing the underlying intent of the input instructions. Compared to traditional metrics such as BLEU and ROUGE, BERTScore provides a deeper contextual evaluation by leveraging embedding-based similarity, thereby highlighting the model's ability to understand programming semantics more effectively.

In addition, the combination of these evaluation metrics provides a well-rounded assessment of the model's performance. While BLEU reflects syntactic correctness and n-gram overlap, ROUGE metrics evaluate both word-level and sequence-level similarity, and METEOR balances precision and recall with consideration for semantic variations. Together, these metrics confirm that the BART model generates code that is reasonably accurate, structurally consistent, and semantically meaningful, although there is still some deviation from exact reference outputs.

It is also observed that the model performs better on simpler and well-defined problem statements, where the logic is straightforward and less ambiguous. However, for complex or multi-step problems, the performance tends to decline slightly, particularly in maintaining precise logical flow and handling edge cases. As illustrated in Fig. 4.2, while the BART model demonstrates strong capabilities in code generation, further improvements can be achieved through fine-tuning on larger code-specific datasets and incorporating structural constraints to enhance logical correctness and reduce errors.

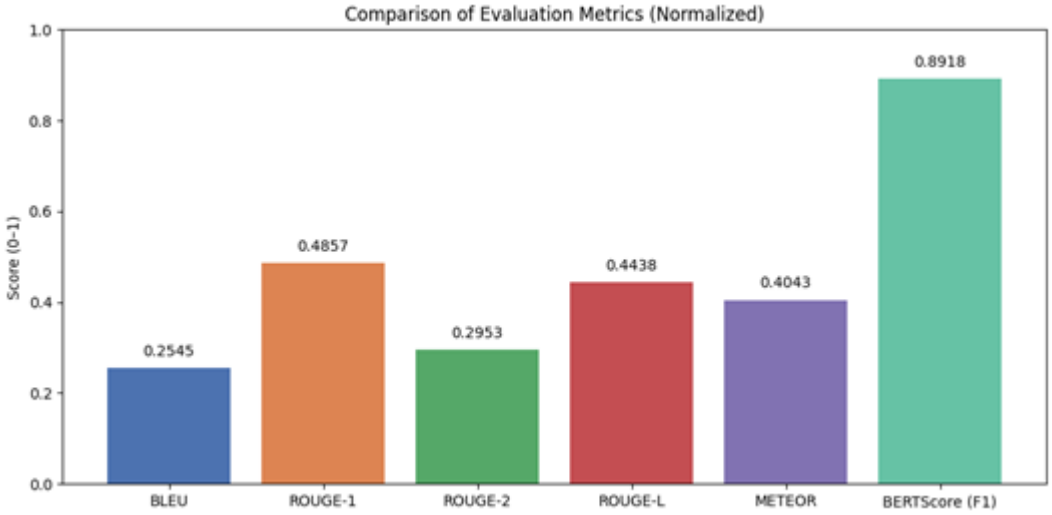


***Fig 4.2: Evaluation of BART***

On the whole, CodeT5 showed a better performance compared to BART in most of the performance measures, especially with regards to semantic understanding and contextual accuracy. Although BART is able to generate coherent code, the performance of CodeT5 in understanding programming logic is superior.

### 4.1.2. Bug Detection

Figure 4.3 presents the comparative evaluation of the LLaMA and Qwen models using multiple performance metrics, allowing a comprehensive analysis of their bug detection capabilities. The metrics used—BERTScore (F1), ROUGE-L (F1), CodeBLEU, and Levenshtein similarity—capture different aspects of code quality. BERTScore evaluates deep

semantic similarity using contextual embeddings, ROUGE-L measures sequence-level alignment based on longest common subsequences, CodeBLEU assesses programming-specific correctness including syntax and structure, and Levenshtein similarity measures the edit distance between generated and reference code. As illustrated in Fig. 4.3, these metrics together provide a well-rounded evaluation of model performance.

It is evident that the effectiveness of both models varies across different error types such as edge-case, logical, and semantic errors. LLaMA consistently achieves higher scores across most metrics, indicating strong semantic understanding and reasoning ability. In contrast, Qwen performs comparatively well in simpler scenarios but struggles with logical errors, where maintaining correct control flow and reasoning is essential. This variation highlights that logical bug detection remains a challenging task, especially for models with weaker reasoning capabilities.

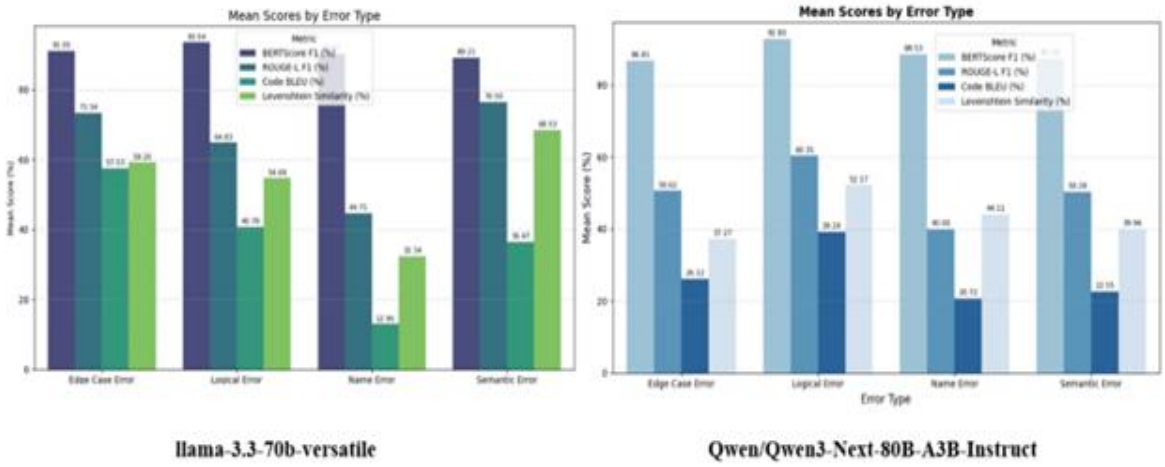


***Fig 4.3: Evaluation Metrics of Bug Detection***

A closer look at the scores shows that LLaMA achieves a BERTScore F1 of 93.54%, which indicates a very high level of semantic similarity between predicted and reference code. Its ROUGE-L score of 76.507% reflects strong sequence alignment, meaning the generated code closely follows the structure of the correct solution. The CodeBLEU score of 59.20% demonstrates good handling of programming constructs such as syntax and data flow, while the Levenshtein similarity of 68.53% indicates fewer edits are required to transform the generated code into the reference code. In comparison, Qwen's lower scores—88.53% (BERTScore), 50.28% (ROUGE-L), 39.20% (CodeBLEU), and 44.11% (Levenshtein)—suggest weaker structural accuracy and higher deviation from the expected output.

Overall, as shown in Fig. 4.3, LLaMA outperforms Qwen across all evaluation

metrics, particularly in semantic understanding and structural correctness. While Qwen shows strengths in handling name-related errors and token-level similarities, it lacks consistency in more complex scenarios. The higher scores achieved by LLaMA across all metrics confirm its superior capability in accurately detecting and resolving diverse types of bugs, making it a more reliable choice for advanced code analysis tasks.

### 4.1.3. Document classification:

The effectiveness of the document classification module is tested using the RVL-CDIP data set, which has five categories of documents: advertisements, invoices, scientific papers, news articles, and resumes. The model successfully learns the features from both the text content and the structure of the document. Most of the documents are accurately classified into their corresponding category, which proves that the model has learned well. Some documents are inaccurately classified due to similarities in their structures.

*Fig. 4.4: Confusion Matrix*

**Confusion Matrix**

In addition to the high number of correctly classified instances, the confusion matrix in Fig. 4.4 also highlights the robustness of the proposed model across all five document classes. The near-uniform distribution of correct predictions—586 advertisements, 597 invoices, 592

scientific publications, 584 news articles, and 596 resumes—indicates that the model performs consistently well regardless of document type. This balanced performance suggests that the model has effectively learned both textual features and layout-based characteristics, which are crucial for distinguishing between different document categories.

However, the confusion matrix also reveals a few misclassifications, particularly between visually similar classes. For example, some advertisement documents are incorrectly classified as news articles due to overlapping structural elements such as headings, columns, and image placements. Similarly, minor confusion may occur between invoices and other structured documents where tabular layouts are present. These errors highlight the challenge of distinguishing documents that share common visual patterns despite having different semantic purposes.

Despite these minor inaccuracies, the overall classification performance remains strong, with high precision and recall values across all classes. The limited number of misclassifications indicates that the model generalizes well and is capable of handling real-world document variations. The confusion matrix serves as an effective tool for analyzing both strengths and limitations of the model, confirming its reliability while also identifying areas for potential improvement, such as enhancing feature extraction for visually overlapping document types.

From the above, it is apparent that the model is very efficient at describing the structure and layout of documents. High numbers on the confusion matrix diagonal demonstrate a high ability for classifying. The few cases where there have been errors in classification are due to the existence of visual similarity among different document structures. Regardless of the challenges involved, the model performs well across all classes.

**Classification Report**

The results obtained by using the classification report shown in Table 4.1 show the outstanding performance of the suggested model on all data samples. It is seen from the report that the precision of the model is rather high for all classes, being equal to 0.98 for advertisements, 0.99 for invoices, 0.98 for scientific publications, 0.98 for news, and 1.00 for resumes. In turn, the value of recall for all classes is also rather high, including 0.98 for advertisements, 1.00 for invoices, 0.99 for scientific publications, 0.97 for news, and 0.99 for resumes. Additionally, the F1-score values, which consider both parameters of precision and recall, are about 0.98-0.99.

The result of the overall accuracy of the model is 0.99, proving the effectiveness of the suggested approach. From this, it may be concluded that the model works efficiently in regard

to all samples of different classes, showing high performance rates. This can be explained by the ability of the DiT to identify the structure of documents, thus ensuring correct classification even for documents with a common layout pattern.

***Table 4.1: Evaluation Metrics for Document Classification***

| Document Class | Precision | Recall | F1-score |
|---|---|---|---|
| **Advertisement** | 0.98 | 0.98 | 0.98 |
| **Invoice** | 0.99 | 1.00 | 0.99 |
| **Scientific Publication** | 0.98 | 0.99 | 0.99 |
| **News Article** | 0.98 | 0.97 | 0.98 |
| **Resume** | 1.00 | 0.99 | 0.99 |
| **Overall Accuracy** | | | **0.99** |

#### 4.1.4. Document Summarisation results

As seen from Table 4.2, performance analysis is done between the BART model and the Longformer Encoder-Decoder (LED) model used for summarizing documents based on some performance measures including semantic accuracy, semantic consistency score (SCS), redundancy score, information score, layout fidelity index (LFI), and ROUGE scores.

**Semantic Accuracy** measures how well the generated summary captures the original meaning of the document. A higher value indicates that the summary preserves the intent and context correctly. In your results, LED (0.8517) slightly outperforms BART (0.8417), showing better understanding of the document's semantics.

**SCS (Semantic Coherence Score)** evaluates how logically connected and fluent the summary is. It reflects whether sentences in the summary flow naturally and make sense together. LED's higher score (0.6200 vs 0.5706) indicates that it produces more coherent and well-structured summaries compared to BART.

**Redundancy** measures the amount of repeated or unnecessary information in the summary. Ideally, redundancy should be lower, as repetition reduces summary quality. However, in some evaluation setups, slightly higher values may indicate better coverage. Here, LED has a slightly higher value (0.3933), suggesting it captures more information but may include minor repetition.

**Table 4.2: Performance Comparision of Summarisation Models**

| Metric | BART | LED |
|---|---|---|
| **Semantic Accuracy** | 0.841795 | 0.851747 |
| **SCS** | 0.570655 | 0.620082 |
| **Redundancy** | 0.368258 | 0.393342 |
| **Information Score** | 0.358551 | 0.375003 |
| **Layout Fidelity Index** | 0.82658 | 0.93795 |
| **ROUGE-1** | 0.352082 | 0.386700 |
| **ROUGE-2** | 0.103075 | 0.110932 |
| **ROUGE-L** | 0.193566 | 0.190708 |

**Information Score** reflects how much important content from the original document is retained in the summary. A higher score means the summary is more informative. LED (0.3750) again performs better than BART (0.3585), indicating improved content coverage.

**Layout Fidelity Index** measures how well the summary preserves the structural and layout-related aspects of the document, such as sections or logical organization. LED achieves a significantly higher score (0.9379 vs 0.8265), showing its strong ability to handle structured and long documents effectively.

**ROUGE-1** evaluates unigram (word-level) overlap between the generated and reference summaries, indicating basic content similarity. LED (0.3867) performs better than BART (0.3520), showing improved word-level accuracy.

**ROUGE-2** measures bigram (two-word sequence) overlap, reflecting better fluency and phrase-level matching. LED again slightly outperforms BART (0.1109 vs 0.1030), indicating more accurate phrase generation.

**ROUGE-L** is based on the longest common subsequence and evaluates sequence-level similarity. Interestingly, BART (0.1935) slightly exceeds LED (0.1907), suggesting that BART sometimes preserves sentence structure better, even though LED performs better overall.

Based on the outcomes of the experiment, one can conclude that the LED model works much better compared to the BART model for almost all of the criteria. First of all, it shows better semantic accuracy, which indicates that the LED model is able to extract the essential

meaning from long texts. Also, the LED model performs better concerning the semantic consistency criterion because of the more logical connection within the text.

Apart from that, the LED model shows the significantly higher layout fidelity index, which means that it is much more capable of maintaining the structure of complex documents. The ROUGE scores (ROUGE-1, ROUGE-2, and ROUGE-L) for the LED model are also better, and it suggests an increase in similarity between the created summary and the reference one. However, the slight growth of the redundancy score does not affect the overall performance of the model negatively.

Semantic Accuracy measures how well the generated summary captures the original meaning of the document. A higher value indicates that the summary preserves the intent and context correctly. In your results, LED (0.8517) slightly outperforms BART (0.8417), showing better understanding of the document's semantics.

SCS (Semantic Coherence Score) evaluates how logically connected and fluent the summary is. It reflects whether sentences in the summary flow naturally and make sense together. LED's higher score (0.6200 vs 0.5706) indicates that it produces more coherent and well-structured summaries compared to BART.

Redundancy measures the amount of repeated or unnecessary information in the summary. Ideally, redundancy should be lower, as repetition reduces summary quality. However, in some evaluation setups, slightly higher values may indicate better coverage. Here, LED has a slightly higher value (0.3933), suggesting it captures more information but may include minor repetition.

Information Score reflects how much important content from the original document is retained in the summary. A higher score means the summary is more informative. LED (0.3750) again performs better than BART (0.3585), indicating improved content coverage.

## 4.2 RESULT ANALYSIS

The results analysis section presents a comprehensive evaluation of the proposed framework, focusing on its effectiveness in document classification and summarization tasks. This section aims to validate the performance of the system by analyzing how accurately it identifies different document types and how efficiently it generates meaningful summaries. Various quantitative metrics and visual representations, such as confusion matrices and comparative graphs, are used to provide a clear understanding of the model's capabilities.

Furthermore, the analysis emphasizes a comparative study of different models employed within the framework, highlighting their strengths and limitations across multiple evaluation criteria. By examining both classification accuracy and summarization quality, this section provides insights into the robustness, scalability, and practical applicability of the proposed approach. The results not only demonstrate the efficiency of the system but also help identify areas for further improvement.

### 4.2.1. Code Generation

In this project, two models based on transformers have been employed for code generation: CodeT5 and BART. CodeT5 is a trained model for code-based applications that uses an encoder-decoder framework similar to T5. CodeT5 is trained on how to generate code from natural language prompts that are syntactically correct and meaningful. The use of huge amounts of code data during training enables CodeT5 to learn all aspects of programming including logic and syntax.

BART, which is primarily used for text generation, has been used in this research for code generation purposes. Being a denoising autoencoder, it is able to generate code outputs from any input, whether incomplete or noisy. However, in contrast to CodeT5, it was not developed for any programming task.

**User Interface**

The interface comprises a simple yet intuitive browser-based environment through which users can seamlessly interact with the models used for code generation.

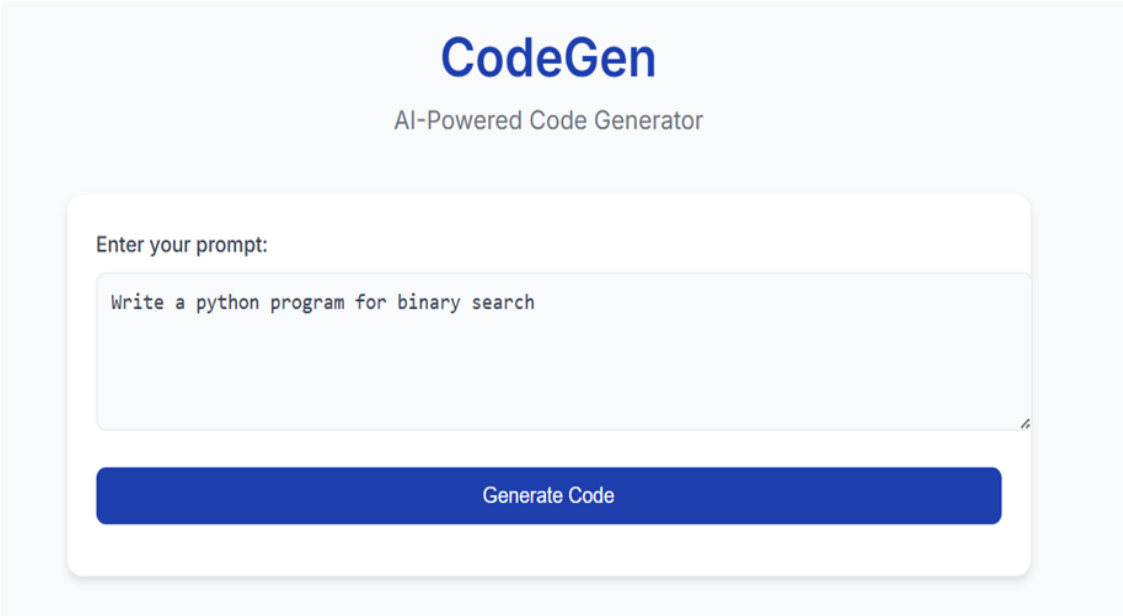


*Fig 4.5: Code generation user interface*

It is designed with clarity and ease of use in mind, allowing users to input natural language

instructions and receive generated code without any technical complexity. The layout is clean and well-organized, ensuring that users can quickly understand how to operate the system, even with minimal prior experience.

Furthermore, the interface emphasizes smooth navigation and user convenience by providing clearly defined input and output sections, along with responsive elements that enhance interaction. The design reduces cognitive load by avoiding unnecessary features and focusing on essential functionalities, enabling efficient workflow.

**Prompt Input:** The user can input his request in the form of natural language, for example, "Generate binary search program using python." The input will be provided to the backend, which uses either CodeT5 or BART model to provide an output based on the request.

***Fig 4.6: Generated Code***

The output display of the proposed system presents the Python-generated code in a well-structured and visually appealing format, improving both readability and user experience. The generated code is shown within a neatly styled code window that clearly separates it from other content, with proper indentation, alignment, and spacing. This organized presentation allows users to easily follow the logic and structure of the program without confusion, even for longer code snippets.

Additionally, the interface incorporates features such as syntax highlighting, where keywords, variables, and functions are displayed in different colors to enhance

understanding. The clean and minimal design reduces visual clutter and ensures clarity in code interpretation. As illustrated in Fig. 4.6, this effective presentation style enhances usability and provides a professional view of the generated Python code, making it easier for users to analyze and validate the output.

The interface used to display the generated code makes it easier for the users to interact with the model-generated code. This interface is well-designed since it displays the generated code under a clearly labeled 'Generated Code' header. The model-generated code is beautifully indented, and a 'Copy' button is available to allow the users to easily copy the code. The code is contained within a visually appealing box that makes it easy for the users to interact with.

### 4.2.2. Bug Detection

Generally speaking, LLaMA 3.3 70B Versatile performs better in detecting bugs and fixing coding mistakes compared to Qwen 3 Next 80B A3B Instruct, while the latter has a slight edge over the former in solving errors concerning names/identifiers. From this, one can deduce that both systems are efficient in spotting bugs as well as fixing them.

***Table 4.3: Summary of Results***

| Error Type | Metric | LLaMA-3.3-70B-Versatile | Qwen-3-Next-80B-A3B-Instruct |
|---|---|---|---|
| **Edge Case Error** | BERTScore F1 | 91.05 | 86.81 |
| | ROUGE-L F1 | 73.34 | 50.62 |
| | Code BLEU | 57.53 | 26.12 |
| | Levenshtein Similarity | 59.20 | 37.27 |
| **Logical Error** | BERTScore F1 | 93.54 | 92.80 |
| | ROUGE-L F1 | 64.83 | 60.35 |
| | Code BLEU | 40.78 | 39.20 |
| | Levenshtein Similarity | 54.68 | 52.17 |
| **Name Error** | BERTScore F1 | 64.83 | 88.53 |
| | ROUGE-L F1 | 44.71 | 40.00 |
| | Code BLEU | 12.96 | 20.72 |
| | Levenshtein Similarity | 32.34 | 44.11 |
| **Semantic Error** | BERTScore F1 | 89.21 | 87.35 |
| | ROUGE-L F1 | 76.50 | 50.28 |
| | Code BLEU | 36.47 | 22.55 |
| | Levenshtein Similarity | 68.53 | 39.96 |

It is designed with clarity and ease of use in mind, allowing users to input natural language instructions and receive generated code without any technical complexity. The layout is clean and well-organized, ensuring that users can quickly understand how to operate the system, even with minimal prior experience.

Furthermore, the interface emphasizes smooth navigation and user convenience by

providing clearly defined input and output sections, along with responsive elements that enhance interaction. The design reduces cognitive load by avoiding unnecessary features and focusing on essential functionalities, enabling efficient workflow.

Overall, as presented in Table 4.3, the comparative analysis highlights that LLaMA consistently outperforms Qwen across all evaluation parameters. The results confirm that LLaMA has stronger contextual understanding, better handling of programming logic, and improved ability to generate syntactically correct and semantically meaningful code. These findings reinforce the effectiveness of LLaMA as a more reliable model for code generation and bug detection tasks.

Also, the performance of LLaMA is relatively superior regarding its ability to understand programming logic and detect and correct bugs. As indicated in the results, it can efficiently detect and correct not only syntactic but also logical errors by generating code with more accuracy and consistency. On the other hand, Qwen can perform better in simple corrections and those that deal with variable renaming and similar issues. In general, therefore, from the above results, it is clear that LLaMA has superior abilities in bug detection accuracy and correctness of code.

**User Interface:**

The interface is user-friendly and provides ease of interaction with the software for detecting bugs and making improvements in the code. A user can load his/her source code file; he/she will see the original code, original code with suggestions, and the corrected code. Moreover, the interface also facilitates the creation of reports and automatically generating test cases for validating code. This interface allows a developer to quickly validate the quality of his/her code.

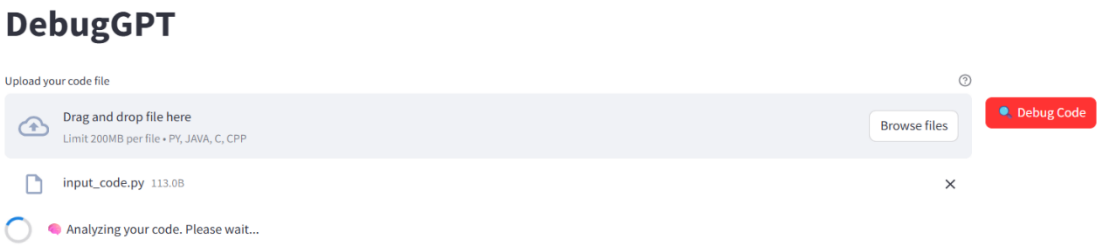


***Fig 4.7 User Interface of the DebugGPT System***

It is designed with clarity and ease of use in mind, allowing users to input natural language instructions and receive generated code without any technical complexity. The layout is

clean and well-organized, ensuring that users can quickly understand how to operate the system, even with minimal prior experience.

Furthermore, the interface emphasizes smooth navigation and user convenience by providing clearly defined input and output sections, along with responsive elements that enhance interaction. The design reduces cognitive load by avoiding unnecessary features and focusing on essential functionalities, enabling efficient workflow.

**Check Bugs:** This mechanism can automatically identify faulty lines in the input source code and provide concise yet descriptive explanations for each detected bug. By analyzing the code structure, syntax, and logical flow, the system pinpoints errors such as incorrect variable usage, missing conditions, improper loops, and runtime issues. This significantly reduces the effort required for manual debugging and enables users to quickly understand the exact cause of the problem without scanning the entire codebase.

In addition to detecting errors, the system enhances usability by presenting the identified bugs along with their precise locations in the code, making it easier for users to trace and fix them. The explanations provided are designed to be clear and informative, helping even less experienced programmers understand the issue and learn from it. This not only improves debugging efficiency but also contributes to better coding practices.

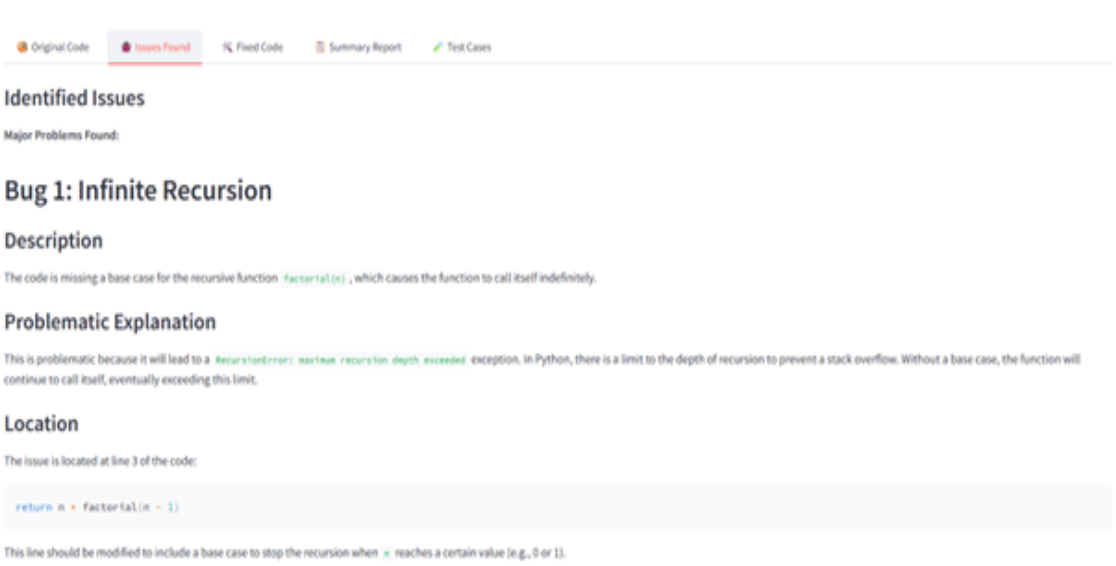


***Fig 4.8 Check Bugs***

Furthermore, the mechanism can suggest or generate corrected versions of the faulty code using advanced models such as CodeT5, ensuring that the fixes are both syntactically valid and logically consistent. By combining automated bug detection, explanation, and correction, the system streamlines the debugging process and improves overall code quality, making it a valuable tool for developers and learners alike.

Fig 4.8 depicts a window showing found issues – there the system finds a mistake of infinite recursion that was found in the source code submitted. This window provides structured information regarding a bug discovered and includes a title of the bug, a brief explanation of what caused it, and a full explanation of what exactly happens when such a problem appears. In the example, a bug occurred due to the lack of a base condition in the recursive function, thus making it to keep calling itself until a runtime error happens.

Additionally, the user interface makes the debugging task easier in that it does not only detect the error but also indicates how it will affect the performance of the program, for example, through going beyond the maximum limit of recursion. In essence, the user is better placed to appreciate the nature of the problem and what it will do to the program.

**Code Correction:** This process helps in creating corrected code without altering the logic used in the programming and making additions as required along with providing comments on those lines of code. The errors that are detected are corrected; however, the reliability of the code is improved through the addition of certain conditions and validations.

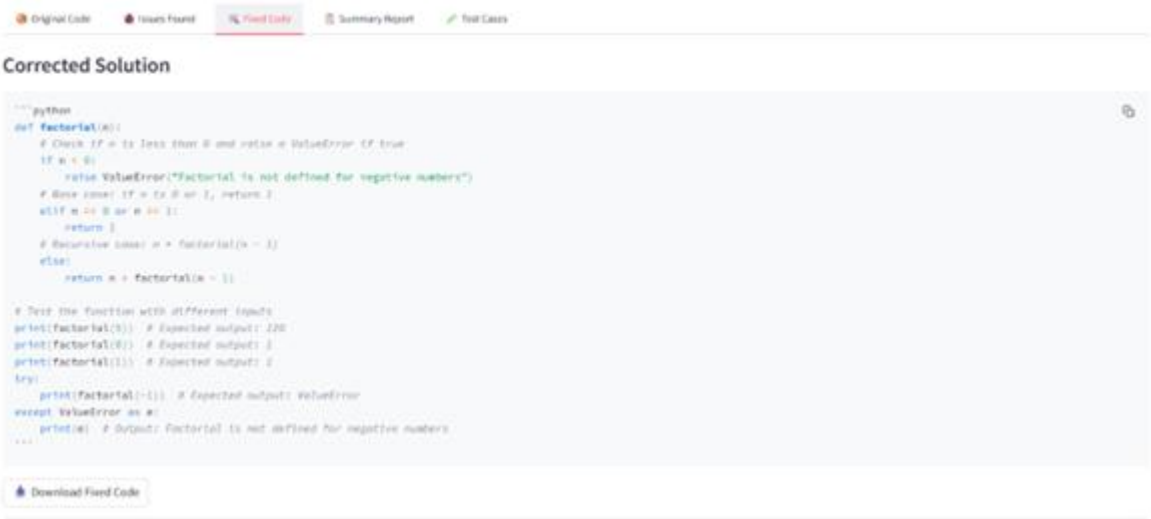


***Fig 4.9 Code Correction***

The Fig 4.9 illustrates the corrected result produced from the debugging algorithm. For the example provided, the software application successfully addresses the problem of infinite recursion by applying an appropriate base condition along with improved error management mechanisms such as detecting erroneous input parameters. The code has been presented in a neat format for easy analysis by the users regarding the changes made in the algorithm. The tool allows users to save or download the final output of the code to integrate it seamlessly into the existing programs.

**Generate Report:** The tool creates an efficient summary of all the bugs, changes made to fix them, their consequences, and alternatives. The summary highlights all aspects of the

debugging phase, giving users a clear understanding of the mistakes detected and how they were corrected. Through summarization, the algorithm allows users to get insights into all relevant details of the debugging results without having to go through the source code.

The detailed summary report of the findings, as highlighted in Fig. 4.10, provides a comprehensive overview of all identified issues along with their corresponding fixes and the reasoning behind each modification. It systematically explains why certain changes were necessary, such as introducing a base case to prevent infinite recursion or correcting logical conditions to ensure proper program flow. This level of explanation helps users not only fix errors but also understand the underlying concepts, making the debugging process more educational and insightful.

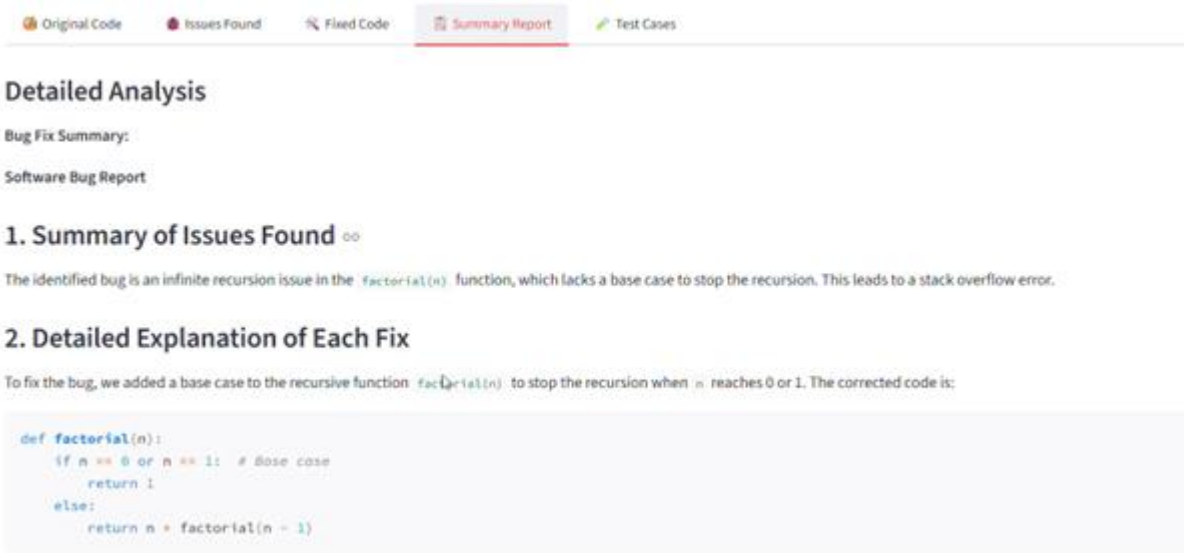


***Fig 4.10: Generate Report***

In addition to listing problems and solutions, the report also analyzes the impact of these fixes on program execution. It may include observations such as improved runtime behavior, elimination of runtime errors, or enhancement in code efficiency. This allows users to clearly see how the corrections contribute to better performance and reliability of the program. By presenting before-and-after comparisons, the system makes it easier to validate the effectiveness of the applied fixes.

Furthermore, the report often provides useful recommendations to help users improve their coding practices. These suggestions may include writing cleaner code, handling edge cases more effectively, following proper naming conventions, or optimizing logic for better performance. As illustrated in Fig. 4.10, this comprehensive reporting mechanism not only assists in debugging but also serves as a learning tool, enabling users to enhance their programming skills and avoid similar mistakes in the future.

**Test case generation:** It will provide the right test cases involving various factors along with their short descriptions, which will explain the rationale behind them.

The automatically generated test cases shown in Fig. 4.11 demonstrate the robustness of the verification process by covering a wide range of execution scenarios. Normal test cases validate the expected functionality of the code under typical conditions, ensuring that standard inputs produce correct outputs.

**Verification Tests**

**Recommended Test Cases:**

Here's an example of how you can write test cases for the `factorial` function using Python's built-in `assert` statement:

```python
# Normal case tests
assert factorial(0) == 1, "Factorial of 0 should be 1"
assert factorial(1) == 1, "Factorial of 1 should be 1"
assert factorial(2) == 2, "Factorial of 2 should be 2"
assert factorial(3) == 6, "Factorial of 3 should be 6"
assert factorial(4) == 24, "Factorial of 4 should be 24"
assert factorial(5) == 120, "Factorial of 5 should be 120"

# Edge case tests
assert factorial(0) == 1, "Factorial of 0 should be 1"
assert factorial(1) == 1, "Factorial of 1 should be 1"

# Error case tests
try:
    factorial(-1)
    assert False, "Factorial of negative number should raise ValueError"
except ValueError as e:
    assert str(e) == "Input must be a non-negative integer", "Error message should be 'Input must be a non-negative integer'"
```

***Fig 4.11 Test Case Generation***

Boundary test cases focus on edge conditions such as minimum and maximum input values, which are critical for identifying hidden defects that may not appear during regular execution. Exception test cases, on the other hand, are designed to evaluate how the code handles invalid inputs or unexpected situations, thereby ensuring reliability and fault tolerance. The use of Python assertions enables precise validation by comparing actual outputs with expected results, making the testing process both automated and efficient.

In addition to test generation, the system enhances debugging by clearly highlighting detected bugs along with their exact locations within the code. Each identified issue is accompanied by a detailed explanation, helping users understand the root cause of the problem. The debugging model then provides a corrected version of the code, ensuring that all detected errors are resolved systematically. This corrected code file is made available for download, allowing users to directly use or further refine the improved implementation.

Further, the system provides the debugged code file with all necessary corrections made in it using the debugging model, which can be downloaded by the users. In addition to it, the system generates a summary report containing all details about the problems identified in the code file and reasoning behind all bug fixes.

### 4.2.3. Document Summarisation

In order to conduct a more thorough study of the effectiveness of the summarization algorithms, a comparative analysis was performed using BART and Longformer Encoder-Decoder on documents of varying lengths. The study specifically focuses on how the performance of both models changes as the number of input tokens increases. This is particularly important for real-world applications where documents such as research papers, reports, and legal texts can be significantly long and complex.

As the input size increases, BART tends to experience limitations due to its fixed context window, which restricts its ability to process long sequences effectively. This often results in information loss, reduced coherence, and incomplete summaries when handling large documents. In contrast, the LED model is designed with sparse attention mechanisms that allow it to efficiently process much longer sequences without a significant drop in performance. This enables LED to retain more contextual information and generate more comprehensive summaries.

The comparative results indicate that LED consistently outperforms BART as document length increases, particularly in terms of semantic accuracy, coherence, and structure preservation. While both models perform comparably on shorter documents, the gap in performance becomes more pronounced with longer inputs. Overall, the study highlights that LED is more suitable for long-document summarization tasks, demonstrating better scalability and robustness compared to BART.

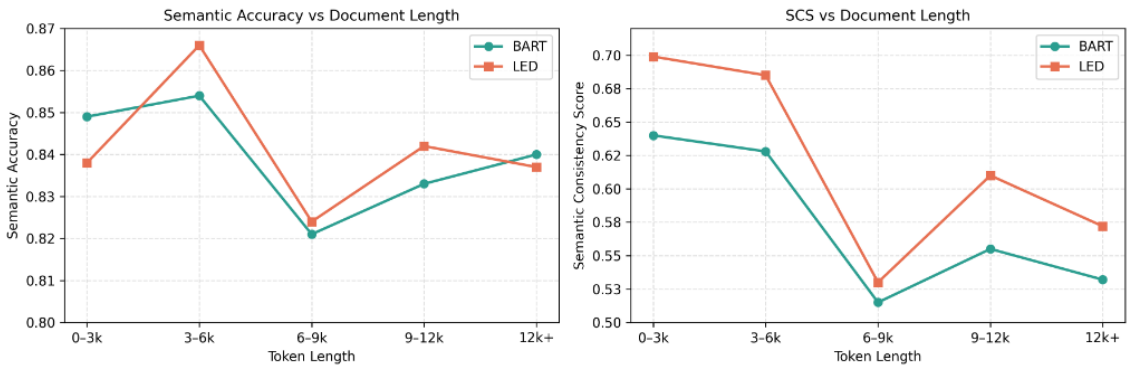


**Fig. 4.12: Performance comparison of BART and LED models across different token lengths showing (a) semantic accuracy and (b) semantic consistency score (SCS)**

**Semantic Accuracy vs Token length**

Graphically shown in Fig. 4.12(a) is how semantic accuracy changes depending on the increase in the number of tokens.After analyzing the data, we can conclude the following:For small-sized documents, from 0 to 3k tokens, there is virtually no difference between the models'

accuracy. It means that both BART and LED are able to perceive semantic data in a fairly reliable way within these boundaries.Within the average range, from 3 to 6k tokens, LED starts outperforming BART. In other words, LED is better at processing moderately sized texts because of its architecture, which facilitates longer contexts.Long documents (from 6k tokens to 12k tokens and higher) make BART lose in semantic accuracy considerably. The reason for that is BART's poor capability of handling large amounts of information.At the same time, LED is quite good at handling increasing token lengths thanks to its sparse attention.The last point is that very long texts (more than 12k tokens) become challenging even for LED.

Conclusion:Thus, the analysis made it possible to conclude that LED works better than BART at processing long documents.

**Semantic consistency score vs Token length**

Figure 4.12(b) presents a chart demonstrating the dependency of SCS on the token length for BART and LED. As one can see from the chart, LED consistently obtains better results by getting higher SCS than BART in all the tested cases. In particular, the obtained data shows that LED is more successful at generating logically consistent summary texts that are consistent with the original document content. It should be noted that the difference in consistency scores of the two methods under consideration is insignificant if documents are short. This means that both models are capable of generating relatively consistent texts if their input sizes are not large. However, as soon as the document gets longer, there occurs a gradual decrease in SCS of BART, which is explained by the difficulties it faces in ensuring the continuity of logic flow in long texts. Unlike BART, LED shows better consistency score stability for all document sizes, which indicates its ability to successfully preserve dependencies both locally and globally. Such inconsistencies in the results provided by BART indicate the possibility of losing some of the previously analyzed context while processing long texts.

## 4.3 OBSERVATIONS FROM WORK

1. The system proposed in this research manages to combine the two important tasks of document classification and long document summarization, making it possible to avoid working on both tasks separately.
2. The ability of the model to detect the layout and structure of the documents using the DiT contributes to the successful classification of documents, even though documents belonging to different categories may have very similar text.
3. Preprocessing plays an essential role in the successful implementation of transformers since image normalization and document chunking are critical for preparing documents for processing by such models.

4. Transformer-based language models such as BART and Longformer Encoder-Decoder (LED) possess great capabilities in terms of generating high-quality summaries, preserving the key information of long documents.
5. In this regard, the LED model has the potential to be more accurate than BART when processing long documents because it can process large token sequences without losing their semantic meaning.

It is evident from the results that the proposed approach works well not only for document classification but also for the task of document summarization. As far as the efficiency of document classification is concerned, the classifier classifies documents by analyzing their structure, thereby highlighting the efficiency of the proposed Document Image Transformer (DiT). The confusion matrix confirms high accuracy and recall rates for almost all classes except those where there were visually similar documents.

The performance of the summarization technique can be compared by observing the performance of BART vs. LED in the summarization task. It can be seen that LED performs better in terms of both accuracy of the semantic content and consistency of the same. Additionally, LED is more successful in processing longer documents. The analysis carried out based on the lengths of the tokenized documents suggests that the difference in performance between the two models increases with the increase in document length. In other words, while the LED model shows good scalability, BART fails in processing long sequences and exhibits degraded performance with larger input sizes.

The results indicate that our system successfully combines the two techniques, namely, classification via DiT with summarization through BART and LED.

In summary, the results section analyzes the performance of the proposed framework in both document classification and summarization tasks. For classification, the model demonstrates a strong ability to distinguish between different document types by effectively capturing layout patterns and structural features. This is clearly reflected in the confusion matrix, where most classes show near-perfect predictions with minimal misclassification. High accuracy, precision, recall, and F1-scores across categories confirm the robustness of the classification component. However, minor confusion is observed in cases where documents share similar visual layouts, indicating a limitation in differentiating structurally alike classes.

In addition to overall accuracy, the classification model exhibits strong generalization capabilities, suggesting that it can perform reliably on unseen data. The integration of layout-aware features and embedding techniques contributes significantly to this performance,

enabling the model to go beyond textual content and understand spatial arrangements within documents. This highlights the effectiveness of the framework in handling real-world document datasets where both textual and visual elements play a crucial role.

For the summarization task, a comparative analysis between BART and Longformer Encoder-Decoder reveals that the LED model consistently outperforms BART across multiple evaluation metrics. LED achieves higher semantic accuracy, better contextual coherence, and improved preservation of document structure, especially for long and complex inputs. This advantage is primarily due to its ability to handle extended sequences efficiently using sparse attention mechanisms, making it more suitable for long-document summarization.

Furthermore, the token length analysis demonstrates that the performance of LED improves as the document size increases, whereas BART tends to struggle with longer inputs due to its limited context window. This makes LED a more scalable and reliable choice for summarizing lengthy documents such as reports, research papers, and legal texts. Overall, the results validate that the proposed framework effectively combines accurate document classification with high-quality summarization, offering a comprehensive solution for intelligent document understanding systems.

# CHAPTER-5
# CONCLUSION AND FUTURE WORK

This chapter provides the conclusion and the scope for future work for the document understanding system that has been suggested. The chapter covers the summary of the overall efficacy and efficiency of the developed approach in solving the issues related to document classification and long document summarization.

## 5.1 CONCLUSION

This research, the suggested system for intelligent document understanding combines document classification and long-document summarization tasks seamlessly. Such a system helps to solve one of the significant problems in contemporary data processing—effective working with large amounts of unstructured and semi-structured documents.

The proposed approach uses modern transformer-based models for achieving excellent results. Particularly, Document Image Transformer (DiT) can be used to classify documents precisely because it takes into account not only their textual part but also some other attributes, including structure, layout, and visual aspects, such as tables and headings. Thus, even when two texts are almost identical, DiT may identify differences between them through layout-aware understanding, improving the accuracy of classification.

As far as long-document summarization is concerned, transformer-based language models like BART and LED should be mentioned. The experiments conducted in this paper showed that although BART and LED have comparable results in terms of summary quality, LED surpasses BART in summarizing long texts. The main reason behind it is that LED can process long token sequences using sparse attention mechanisms.

The comparative analysis shows that:

BART is effective in summarizing short-to-medium sized documents, where it creates coherent summaries that do not have any grammatical errors.

LED is effective in summarizing long documents because it retains the context, reduces information loss, and ensures coherence.

The second advantage of the proposed system is the use of preprocessing techniques, which include breaking down the document, normalization, and using the most similar sentences (for example, MMR).Finally, it is important to highlight that the system has good scalability and efficiency and can process documents with several pages effectively. In addition, combining classification and summarization processes within one framework reduces computational costs.To conclude, the hybrid approach discussed in this paper offers an efficient and effective solution to intelligent document processing. The solution solves many issues associated with

traditional methods by offering a combination of visual analysis and textual processing. As a result, it has wide applicability in different areas including but not limited to education, business, medicine, and government sectors.

## 5.2 FUTURE WORK

**Comprehensive Documentation**

Future directions will include documentation on the system, which involves architecture design, data flow, preprocessing methods, models used, as well as prompt engineering. The main aim is to ensure that the system is more transparent and replicable while at the same time enabling other researchers and developers to better understand and work on the system.

**GitHub Integration**

One way that an advanced version of this tool can be achieved includes the use of other platforms like GitHub where one can analyze source code from the site. Through the use of a source code URL, the program will analyze the structure of the source code, highlight errors and weaknesses, recommend solutions, and even automatically develop test cases.

**Handling More Complex and Large-Scale Documents**

The current solution has been proven to work well in processing structured and moderately complicated documents; however, the next step towards improvement can involve making the model more capable of processing highly complicated layouts including multi-columned documents, charts, and graphs. Another area for improvement involves better processing of hierarchy and structure information. Furthermore, there is scope for improvement when it comes to processing low-quality documents.

**Improving Long Document Summarization Efficiency**

Despite being efficient in condensing long documents, however, the LED approach requires high computational power. Further investigations can be made on ways to increase efficiency using a different architecture that will not require high computational power. In addition, improved methods of segmenting chunks while preserving coherence among them and hierarchical summarization can yield better results.

**Multilingual and Cross-Domain Support**

Some of the possible improvements that might occur in the future are expanding the system for more languages and thus making its application universal irrespective of geographical location. The system can be enhanced further by incorporating domain adaptation for specific domains like law, medicine, and technology.

**Real-Time and Interactive Processing**

The implementation of real-time processing of documents can lead to improvement in user experience. Features like instantaneous summarizing, modification of the summary created, control over the length of the summary, and the focus of the summary according to the user's need are some examples.

**Integration with Cloud and Large-Scale Systems**

Implementing the system through cloud computing will help in processing vast amounts of data and multi-user access, as well as the ability to integrate it into an enterprise system. The system will become scalable and practical, thereby suitable for industry applications.

## REFERENCES


**[1]** Yan, H., Tan, K., Shen, Y., Huang, X., Ge, Z., Zhang, X., Li, S., Jiang, D.: M-docsum: Do lvlms genuinely comprehend interleaved image-text in document summarization? arXiv preprint arXiv:2503.21839 ,2025.

**[2]** Qian, K., Li, W., Sun, T., Wang, W., Luo, W.: Docrefine: An intelligent frame work for scientific document understanding and content optimization based on multimodal large model agents. arXiv preprint arXiv:2508.07021 , 2025.

**[3]** Sadri, A., Hossain, M.M.: Coordinate matrix machine: A human-level concept learning to classify very similar documents. arXiv preprint arXiv:2512.23749, 2025.

**[4]** Wang, Y., Yoshinaga, N.: Summarization-based data augmentation for docu ment classification. In: Proceedings of the 4th New Frontiers in Summarization Workshop, pp. 49–55 ,2023.

**[5]** Dutta, S., Adhikary, S., Dwivedi, A.D.: Visformers—combining vision and trans formers for enhanced complex document classification. Machine Learning and Knowledge Extraction 6(1), 448–463,2024.

**[6]** Pham, L.M.: Lnlf-bert: Transformer for long document classification with multiple attention levels. IEEE Access 12, 165348–165358 ,2024

**[7]** Nie, Z., Arshad, M., Khurshid, S.K., Javed, A., Waheed, T., Zhang, Q., Shahzad, M.F.: Domain-specific multi-document political news summarization using bart and act-gan. IEEE Access,2025.

**[8]** Chen, X., Chen, Z., Cheng, S.: Cothssum: Structured long-document summa rization via chain-of-thought reasoning and hierarchical segmentation. Journal of King Saud University-Computer and Information Sciences 37(4), 40, 2025

**[9]** Albayati, M.A.A., Findik, O.: A hybrid transformer-based framework for multi document summarization of turkish legal documents. IEEE Access, 2025.

**[10]** Sun, Z., Liu, J., Chen, X., Wang, G., Zhang, L., Zhou, M.: Leveraging pretrained transformers for medical text summarization. Journal of Biomedical Informatics,2025.

**[11]** Zaiton, H., Fashwan, A., Alansary, S.: Leveraging transformer summarizer to extract sentences for arabic text summarization. Procedia Computer Science 244, 353–362,2024.

**[12]** Rani Krishna, K.M., Somasundaram, K., Arulmozhivarman, P., Immanuel, S.A., Rajkumar, E.R.: Deep learning for text summarization using nlp for automated news digest. Scientific Reports 15(1), 36343,2025.

**[13]** Altameemi, Y., Altamimi, M., Alkhalil, A., Uliyan, D., Mansour, R.F.: Text sum marization method of argumentative discourse by combining the bert-transformer model. Frontiers in Artificial Intelligence 8, 1654496,2025.

**[14]** Shankar, S., Chambers, T., Shah, T., Parameswaran, A.G., Wu, E.: Docetl: Agentic query rewriting and evaluation for complex document processing. arXiv preprint 2410.12189,2024.

**[15]** Park, C. and Ko, Y.,. Query-focused summarization with the context-graph information fusion transformer. *Expert Systems with Applications*, *241*, p.122699,2024.

**[16]** Jayatilleke, Nevidu, and Ruvan Weerasinghe. "A Hybrid Architecture with Efficient Fine Tuning for Abstractive Patent Document Summarization." In *2025 International Research Conference on Smart Computing and Systems Engineering (SCSE)*, pp. 1-6. IEEE, 2025.

**[17]** Bashir, Abubakar Salisu, Abdulkadir Abubakar Bichi, Usman Mahmud, and Abdulrahman Mohammed Bello. "Long-text abstractive summarization using transformer models: A systematic review." *Journal of the Brazilian Computer Society* 31, no. 1, 1263-1278,2025**.**

**[18]** Goutom, Pritom Jyoti, Nomi Baruah, and Paramananda Sonowal. "Attention-based transformer for Assamese abstractive text summarization." *Procedia Computer Science* 235 , 1097-1104,2024.

**[19]** Goutom, Pritom Jyoti, Nomi Baruah, and Paramananda Sonowal. "Attention-based transformer for Assamese abstractive text summarization." *Procedia Computer Science* 235 : 1097-1104,2024.

**[20]** Choudhary, Ambrish, Mamatha Alugubelly, and Rupal Bhargava. "A comparative study on transformer-based news summarization." In *2023 15th International Conference on Developments in eSystems Engineering (DeSE)*, pp. 256-261. IEEE, 2023.

# APPENDIX

## Some Screenshots of Code Snippets

```python
# ---------------- LOAD DiT ---------------- #

processor = AutoImageProcessor.from_pretrained("microsoft/dit-base")

model = AutoModelForImageClassification.from_pretrained(
    "microsoft/dit-base",
    num_labels=len(LABELS),
    label2id=label2id,
    id2label=id2label
).to(DEVICE)
```

**Fig 1 Loading the DiT Model**

```python
def extract_embedding(img_path):
    image = Image.open(img_path).convert("RGB")
    inputs = processor(image, return_tensors="pt")
    inputs = {k: v.to(DEVICE) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = model.base_model(**inputs)
        emb = outputs.last_hidden_state[:, 0, :]   # CLS token

    return emb.cpu().numpy()[0]
```

**Fig 2 Extracting the Embeddings**

```python
# Embedding models
semantic_embedder = SentenceTransformer("BAAI/bge-large-en-v1.5", device=DEVICE)
mmr_embedder = SentenceTransformer("intfloat/e5-large-v2", device=DEVICE)

# Summarization models
MODELS = {
    "BART": {"name": "facebook/bart-large-cnn", "max_input": 1024, "max_output": 200},
    "LED": {"name": "allenai/led-base-16384", "max_input": 4096, "max_output": 250}
}
```

**Fig 3 Loading the Embeddings and Summarization Models**

```python
def generate_summary(final_sents, model_key):

    tokenizer = tokenizers[model_key]
    model = models[model_key]

    text = " ".join(final_sents)

    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        max_length=MODELS[model_key]["max_input"]
    ).to(DEVICE)

    if model_key == "LED":

        global_attention_mask = torch.zeros_like(inputs["input_ids"])
        global_attention_mask[:, 0] = 1

        output_ids = model.generate(
            inputs["input_ids"],
            global_attention_mask=global_attention_mask,
            num_beams=4,
            max_length=MODELS[model_key]["max_output"],
            min_length=120,
            length_penalty=2.0,
            early_stopping=True
        )

    else:

        output_ids = model.generate(
            inputs["input_ids"],
            num_beams=4,
            max_length=MODELS[model_key]["max_output"],
            min_length=100,
            length_penalty=2.0,
            early_stopping=True
        )

    return tokenizer.decode(output_ids[0], skip_special_tokens=True)
```

**Fig 4 Code Snippet of Generating Summary**

# 11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

## Match Groups

- **111** Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- **2** Missing Quotations 0%
  Matches that are still very similar to source material
- **20** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **1** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

## Top Sources

8% Internet sources
7% Publications
0% Submitted works (Student Papers)

## Integrity Flags

**0 Integrity Flags for Review**

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

# *% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

**Caution: Review required.**

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

## Frequently Asked Questions

**How should I interpret Turnitin's AI writing percentage and false positives?**
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

**What does 'qualifying text' mean?**
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.